

Universidade Federal de Viçosa
Campus Rio Paranaíba

John Kauan Oliveira Passos - 8149

Análise Comparativa de Algoritmos de Ordenação com Foco na Complexidade
Computacional

Rio Paranaíba - MG
2025

Universidade Federal de Viçosa
Campus **Rio Paranaíba**

John Kauan Oliveira Passos - 8149

**Análise Comparativa de Algoritmos de Ordenação com Foco na Complexidade
Computacional**

Trabalho apresentado para obtenção de créditos na disciplina SIN213 - Projeto de Algoritmo da Universidade Federal de Viçosa - Campus de Rio Paranaíba, ministrada pelo Professor Pedro Moisés de Souza.

Rio Paranaíba - MG

2025

1 Resumo

Este trabalho apresenta um estudo detalhado sobre os principais algoritmos de ordenação utilizados em computação: *Insertion Sort*, *Bubble Sort*, *Selection Sort*, *Shell Sort*, *Merge Sort*, *Quick Sort* e *Heap Sort (Min-Heap)*. Para cada algoritmo, são discutidos seus princípios de funcionamento, pseudocódigos, análises de complexidade e comportamento empírico através de experimentos realizados com vetores de diferentes tamanhos e distribuições (crescente, decrescente e randômica).

Além disso, são apresentados gráficos e tabelas de desempenho que ilustram, de maneira objetiva, o comportamento temporal de cada método. As análises permitem comparar o desempenho teórico com os resultados observados experimentalmente, evidenciando como cada algoritmo reage ao aumento do tamanho da entrada e às diferentes organizações iniciais dos dados.

Este estudo busca consolidar o entendimento dos mecanismos internos de cada técnica de ordenação, destacando suas vantagens, limitações e adequações práticas. Os resultados mostram, principalmente, a diferença entre algoritmos quadráticos e algoritmos de complexidade $O(n \log n)$, reforçando a importância da escolha adequada da estratégia de ordenação para diferentes contextos computacionais.

...

Sumário

1	Resumo	1
2	Introdução	5
3	Algoritmos	6
3.1	<i>Insertion Sort</i>	6
3.1.1	Funcionamento do <i>Insertion Sort</i>	6
3.1.2	Passo a Passo do <i>Insertion Sort</i>	6
3.1.3	Pseudocódigo do <i>Insertion Sort</i>	7
3.1.4	Teste de Mesa do <i>Insertion Sort</i>	8
3.2	<i>Selection Sort</i>	9
3.2.1	Funcionamento do <i>Selection Sort</i>	9
3.2.2	Passo a Passo do <i>Selection Sort</i>	9
3.2.3	Pseudocódigo do <i>Selection Sort</i>	9
3.2.4	Teste de Mesa do <i>Selection Sort</i>	10
3.3	<i>Shell Sort</i>	11
3.3.1	Funcionamento do <i>Shell Sort</i>	11
3.3.2	Passo a Passo do <i>Shell Sort</i>	11
3.3.3	Pseudocódigo do <i>Shell Sort</i>	12
3.3.4	Teste de Mesa do <i>Shell Sort</i>	13
3.4	<i>Bubble Sort</i>	14
3.4.1	Funcionamento do <i>Bubble Sort</i>	14
3.4.2	Passo a Passo do <i>Bubble Sort</i>	14
3.4.3	Pseudocódigo do <i>Bubble Sort</i>	14
3.4.4	Teste de Mesa do <i>Bubble Sort</i>	15
3.5	<i>Merge Sort</i>	15
3.5.1	Funcionamento do <i>Merge Sort</i>	15
3.5.2	Passo a Passo do <i>Merge Sort</i>	16
3.5.3	Pseudocódigo do <i>Merge Sort</i>	17

3.5.4	Teste de Mesa do <i>Merge Sort</i>	18
3.6	<i>Quick Sort</i>	19
3.6.1	Funcionamento do <i>Quick Sort</i>	19
3.6.2	Passo a Passo do <i>Quick Sort</i>	19
3.6.3	Pseudocódigo do <i>Quick Sort</i>	20
3.6.4	Teste de Mesa do <i>Quick Sort</i>	21
3.7	<i>Heap Sort</i>	22
3.7.1	Funcionamento do <i>Heap Sort (Min-heap)</i>	22
3.7.2	Passo a Passo do <i>Heap Sort</i>	22
3.7.3	Pseudocódigo do <i>Heap Sort (Min-Heap)</i>	23
3.7.4	Teste de Mesa do <i>Heap Sort (Min-Heap)</i>	25
4	Análise de Complexidade	26
4.1	<i>Insertion Sort</i>	26
4.1.1	Melhor Caso - <i>Insertion Sort</i>	26
4.1.2	Médio Caso - <i>Insertion Sort</i>	27
4.1.3	Pior Caso - <i>Insertion Sort</i>	27
4.2	<i>Selection Sort</i>	28
4.2.1	Melhor Caso - <i>Selection Sort</i>	28
4.2.2	Médio Caso - <i>Selection Sort</i>	29
4.2.3	Pior Caso - <i>Selection Sort</i>	30
4.3	<i>Shell Sort</i>	31
4.3.1	Melhor Caso - <i>Shell Sort</i>	31
4.3.2	Médio Caso - <i>Shell Sort</i>	32
4.3.3	Pior Caso - <i>Shell Sort</i>	32
4.4	<i>Bubble Sort</i>	32
4.4.1	Melhor Caso - <i>Bubble Sort</i>	32
4.4.2	Médio Caso - <i>Bubble Sort</i>	33
4.4.3	Pior Caso - <i>Bubble Sort</i>	34
4.5	<i>Merge Sort</i>	35

4.5.1	Melhor Caso - <i>Merge Sort</i>	36
4.5.2	Médio Caso - <i>Merge Sort</i>	37
4.5.3	Pior Caso - <i>Merge Sort</i>	37
4.6	<i>Quick Sort</i>	37
4.6.1	Melhor Caso - <i>Quick Sort</i>	37
4.6.2	Médio Caso - <i>Quick Sort</i>	38
4.6.3	Pior Caso - <i>Quick Sort</i>	39
4.7	<i>Heap Sort</i>	40
4.7.1	Melhor Caso - <i>Heap Sort (Min-Heap)</i>	41
4.7.2	Médio Caso - <i>Heap Sort (Min-Heap)</i>	41
4.7.3	Pior Caso - <i>Heap Sort (Min-Heap)</i>	41
5	Tabela e Gráfico	43
5.1	<i>Insertion Sort</i>	43
5.2	<i>Selection Sort</i>	44
5.3	<i>Shell Sort</i>	45
5.4	<i>Bubble Sort</i>	47
5.5	<i>Merge Sort</i>	48
5.6	<i>Quick Sort</i>	49
5.6.1	Versão 1 (Primeiro elemento como pivô)	49
5.6.2	Versão 2 (Utilização da média para escolha do pivô)	51
5.6.3	Versão 3 (Pivô selecionado de forma randômica)	52
5.7	<i>Heap Sort (Min-Heap)</i>	53
5.8	Comparação dos Algoritmos	55
5.8.1	Crescente	55
5.8.2	Decrescente	57
5.8.3	Randômico	58
6	Conclusão	60
	Referências	61

2 Introdução

A organização eficiente de informações é uma necessidade constante em diversas situações do dia a dia. No âmbito da computação, a ordenação de dados desempenha um papel crucial, pois facilita o processamento e a manipulação de grandes volumes de informações. Um exemplo prático é a busca por um contato em uma lista telefônica; se os nomes não estiverem organizados em ordem alfabética, localizar uma pessoa específica se torna muito mais difícil.

Existem vários métodos de ordenação, cada um com características próprias que afetam seu desempenho conforme o tipo e o tamanho dos dados. Alguns algoritmos são mais eficazes em conjuntos pequenos ou parcialmente ordenados, enquanto outros são projetados para lidar com grandes volumes de informações e exigem alta eficiência computacional.

O objetivo deste estudo é analisar o desempenho de diferentes algoritmos de ordenação em distintas configurações de dados. Para isso, os dados de entrada serão organizados de três formas: ordem crescente, ordem decrescente e aleatória, com o número de elementos variando de 10 a 1.000.000. Dessa maneira, será possível avaliar o tempo de execução e o consumo de recursos computacionais em cada cenário.

Além de examinar o desempenho de cada algoritmo individualmente, este estudo também pretende compará-los, destacando suas aplicações práticas e limitações. Com base nessa análise, espera-se fornecer informações que auxiliem na escolha do método mais adequado para diferentes situações.

...

3 Algoritmos

3.1 *Insertion Sort*

O *Insertion Sort* é um algoritmo de ordenação conhecido por sua simplicidade e eficiência em listas pequenas ou parcialmente ordenadas. Ele se baseia na ideia de construir uma lista ordenada elemento por elemento, inserindo cada novo valor em sua posição correta. Esse processo é comparável a organizar cartas em uma mão de baralho.[1]

3.1.1 Funcionamento do *Insertion Sort*

O algoritmo percorre a lista da esquerda para a direita. Cada elemento é comparado com os anteriores e deslocado até que seja encontrada sua posição correta. Esse procedimento se repete até que todos os elementos estejam ordenados.

3.1.2 Passo a Passo do *Insertion Sort*

1. Considere que o primeiro elemento esteja na parte ordenada.
2. Para cada elemento da lista (a partir do segundo), compare-o com os elementos anteriores da parte ordenada.
3. Desloque os maiores valores uma posição à frente.
4. Insira o elemento na posição correta.
5. Repita o processo até que toda a lista esteja ordenada.

3.1.3 Pseudocódigo do *Insertion Sort*

<i>INSERTION-SORT</i> (A,N)			
	PSEUDOCÓDIGO	CUSTO	VEZES
1-	FOR j=2 TO comprimento(A)	c_1	n
2-	DO CHAVE = A[j]	c_2	$(n - 1)$
3-	inserir A[j] na sequencia ordenada A[1..j-1]	c_3	$(n - 1)$
4-	i = j-1	c_4	$(n - 1)$
5-	WHILE i>0 e A[i] > CHAVE	c_5	$\sum_{j=2}^n t_j$
6-	DO A[i+1] = A[i]	c_6	$\sum_{j=2}^n (t_j - 1)$
7-	i = i-1	c_7	$\sum_{j=2}^n (t_j - 1)$
8-	A[i+1] = CHAVE	c_8	$(n - 1)$

Figura 1: Pseudocódigo do *Insertion-Sort* com análise de custo e vezes.

3.1.4 Teste de Mesa do *Insertion Sort*

entredo					
	5	2	4	6	3
	1	2	3	4	5
WHILE $i > 0$ e $A[i] > CHAVE$					
1	$CHAVE \leftarrow A[1]$	$i \leftarrow i - 1$	$A[i+1] \leftarrow A[i]$	$i \leftarrow i - 1$	$A[i+1] \leftarrow CHAVE$
2	$CHAVE \leftarrow 2$	$i \leftarrow 1$	$A[2] \leftarrow A[1]$	$i \leftarrow 0$	$A[1] \leftarrow 2$
			$A[2] \leftarrow 5$		2 5 4 6 3 3 1 2 3 4 5 6
3	$CHAVE \leftarrow 4$	$i \leftarrow 2$	$A[3] \leftarrow A[2]$	$i \leftarrow 1$	$A[2] \leftarrow 4$
			$A[3] \leftarrow 5$		2 4 5 6 3 3 1 2 3 4 5 6
4	$CHAVE \leftarrow 6$	$i \leftarrow 3$			$A[4] \leftarrow 6$
					2 4 5 6 3 3 1 2 3 4 5 6
5	$CHAVE \leftarrow 1$	$i \leftarrow 4$	$A[5] \leftarrow A[4]$	$i \leftarrow 3$	$A[5] \leftarrow 6$
			$A[4] \leftarrow A[3]$	$i \leftarrow 2$	$A[4] \leftarrow 5$
			$A[3] \leftarrow A[2]$	$i \leftarrow 1$	$A[3] \leftarrow 4$
			$A[2] \leftarrow A[1]$	$i \leftarrow 0$	$A[2] \leftarrow 2$
					2 2 4 5 6 3 1 2 3 4 5 6
					$A[1] \leftarrow 1$
					1 2 4 5 6 3 1 2 3 4 5 6
6	$CHAVE \leftarrow 3$	$i \leftarrow 5$	$A[6] \leftarrow A[5]$	$i \leftarrow 4$	$A[6] \leftarrow 6$
			$A[5] \leftarrow A[4]$	$i \leftarrow 3$	$A[5] \leftarrow 5$
			$A[4] \leftarrow A[3]$	$i \leftarrow 2$	$A[4] \leftarrow 4$
					1 2 4 4 5 6 1 2 3 4 5 6
					$A[3] \leftarrow 3$
					1 2 3 4 5 6

Figura 2: Teste de mesa manuscrito do *Insertion-sort*.

3.2 *Selection Sort*

O *Selection Sort* é um algoritmo de ordenação baseado na seleção repetida do menor (ou maior) elemento e sua troca com a posição inicial da parte não ordenada da lista. Ele é simples, mas pouco eficiente em listas grandes, pois sempre percorre toda a lista restante.[3]

3.2.1 Funcionamento do *Selection Sort*

O algoritmo divide a lista em duas partes: a ordenada e a não ordenada. Repetidamente, ele procura o menor elemento da parte não ordenada e o troca com o primeiro elemento dessa parte, expandindo assim a sublista ordenada.

3.2.2 Passo a Passo do *Selection Sort*

1. Encontre o menor elemento da lista.
2. Troque-o com o primeiro elemento.
3. Em seguida, encontre o segundo menor elemento e troque-o com o segundo elemento da lista.
4. Repita o processo até que todos os elementos estejam ordenados.

3.2.3 Pseudocódigo do *Selection Sort*

<i>SELECTION-SORT</i> (A,N)		
PSEUDOCÓDIGO	CUSTO	VEZES
1- FOR i=1 TO n-1	c_1	$(n - 1)$
2- min = i	c_2	$(n - 1)$
3- FOR j=i+1 TO n	c_3	$\sum_{i=1}^{n-1} (n - 1)$
4- IF A[j] < A[min]	c_4	$\sum_{i=1}^{n-1} (n - 1)$
5- min = j	c_5	$\leq \sum_{i=1}^{n-1} (n - 1)$
6- TROCA A[i] < - > A[min]	c_6	$(n - 1)$

Figura 3: Pseudocódigo do *Selection-Sort* com análise de custo e vezes.

3.2.4 Teste de Mesa do *Selection Sort*

$$\begin{array}{c}
 1 \ 2 \ 3 \ 4 \ 5 \\
 2 \ 5 \ 4 \ 3 \ 1
 \end{array}$$

D	S	T	Q	Q	S
D	L	M	M	J	V

+ troca

i	min=i	j=i+1	if A[j] < A[min]	min=j	A[i] ↔ A[min]	
1	min=1	j=2	A[2] < A[1] 5 < 2 não	min=2	A[1] ↔ A[2] 5 ↔ 2	
2	min=2	j=3	A[3] < A[2] 4 < 5	min=3	A[2] ↔ A[3] 5 ↔ 4	2 4 5 1 3 1 2 3 4 5
3	min=3	j=4	A[4] < A[3] 3 < 5	min=4	A[3] ↔ A[4] 5 ↔ 3	2 4 1 5 3 1 2 3 4 5
4	min=4	j=5	A[5] < A[4] 3 < 5	min=5	A[4] ↔ A[5] 5 ↔ 3	2 4 1 3 5 1 2 3 4 5
5	min=1	j=2	A[2] < A[1] 4 < 2 não			
2	min=2	j=3	A[3] < A[2] 3 < 4	min=3	A[2] ↔ A[3] 4 ↔ 3	2 1 4 3 5 1 2 3 4 5
3	min=3	j=4	A[4] < A[3] 3 < 4	min=4	A[3] ↔ A[4] 4 ↔ 3	2 1 3 4 5 1 2 3 4 5
4	min=4	j=5	A[5] < A[4] 5 < 4 não			
5	min=1	j=2	A[2] < A[1] 1 < 2	min=2	A[1] ↔ A[2] 2 ↔ 1	1 2 3 4 5 1 2 3 4 5

OK

Figura 4: Teste de mesa manuscrito do *Selection-sort*.

3.3 *Shell Sort*

O *Shell Sort*, proposto por Donald Shell em 1959, é uma generalização do *Insertion Sort* que permite a troca de elementos distantes. Ele melhora o desempenho reduzindo o número de deslocamentos necessários.[2]

3.3.1 Funcionamento do *Shell Sort*

O algoritmo começa ordenando elementos distantes uns dos outros e gradualmente reduz o intervalo entre eles (divide lista em sublistas) e aplica o *Insertion Sort* a cada sublista. Essa estratégia melhora a eficiência do *Insertion Sort* aplicado em intervalos menores, resultando em uma ordenação mais rápida.

3.3.2 Passo a Passo do *Shell Sort*

1. Defina uma sequência de intervalos (geralmente metade do tamanho da lista, reduzindo pela metade a cada passo).
2. Ordene os elementos separados por esse intervalo usando *Insertion Sort*.
3. Reduza o intervalo e repita o processo.
4. Continue até que o intervalo seja 1, realizando uma última ordenação.
5. Ao final, a lista estará ordenada.

3.3.3 Pseudocódigo do *Shell Sort*

<i>SHELL-SORT</i> (A,N)		
PSEUDOCÓDIGO	CUSTO	VEZES
1- $h = \lfloor n/2 \rfloor$	c_1	1
2- WHILE $h \geq 1$	c_2	$\log n$
3- FOR $i=h+1$ TO n	c_3	$\sum_{i=1}^{\log n} (n - hi)$
4- chave = $A[i]$	c_4	$\sum_{i=1}^{\log n} (n - hi)$
5- $k=i-h$	c_5	$\sum_{i=1}^{\log n} (n - hi)$
6- WHILE $(k>0)$ AND $(A[k]>chave)$	c_6	$T, T \leq \sum_{i=1}^{\log n} (n - hi)$
7- $A[k+h]=A[k]$	c_7	T
8- $k=k-h$	c_8	T
9- $A[k+h]=chave$	c_9	$\sum_{i=1}^{\log n} (n - hi)$
10- $h = \lfloor h/2 \rfloor$	c_{10}	$\log n$

Figura 5: Pseudocódigo do *Shell-Sort* com análise de custo e vezes.

3.3.4 Teste de Mesa do *Shell Sort*

4, 2, 3, 1

$h = n/2$	$n \geq 1$	$i = h+1$	WHILE $K > 0$	$A[K+h] = A[K]$	$A[K+h] = CHAVE$
		$CHAVE = A[i]$	$A[K] > CHAVE$	$K = K - h$	
		$K = i - h$			
$h = 2$	$2 \geq 1$ OK	$i = 3$	$J > 0$	$A[3] = A[1]$	$A[1] = A[3]$
		$CHAVE = A[3]$	$A[1] > A[3]$	$A[3] = 4$	$A[1] = 4$
		$K = 1$	$4 > 3$ sim	$K = -1$	$A[1] = 3$
			$-1 > 0$ não		$A[3, 2, 4, 1]$
		$i = 4$	$2 > 0$	$A[4] = A[2]$	
		$CHAVE = A[4]$	$A[2] > A[4]$	$A[4] = 2$	$A[2] = A[4]$
		$K = 2$	$2 > 1$ sim	$K[3, 2, 4, 2]$	$A[2] = 1$
			$0 > 0$ não	$K = 0$	$A[3, 1, 4, 2]$
$h = 1$	$1 \geq 1$ OK	$i = 2$	$J > 0$	$A[2] = A[1]$	$A[1] = A[2]$
		$CHAVE = A[2]$	$A[1] > A[2]$	$A[2] = 3$	$A[1] = 1$
		$K = 1$	$3 > 1$ sim	$A[3, 3, 4, 2]$	$A[1, 3, 4, 2]$
			$0 > 0$ não	$K = 0$	
		$i = 3$	$2 > 0$		
		$CHAVE = A[3]$	$A[2] > A[3]$		$A[1, 3, 4, 2]$
		$K = 2$	$3 > 4$ não		
		$i = 4$	$3 > 0$	$A[4] = A[3]$	
		$CHAVE = A[4]$	$A[3] > A[4]$	$A[4] = 4$	
		$K = 3$	$4 > 3$ sim	$A[1, 3, 4, 4]$	
			$2 > 0$	$K = 2$	
			$A[2] > 2$	$A[3] = A[2]$	
			$3 > 2$ sim	$A[3] = 3$	
				$A[1, 3, 3, 4]$	
			$J > 0$	$K = 1$	
			$A[1] > 2$ não		$A[2] = 2$
					$A[2] = 2$
					$A[1, 2, 3, 4]$

Figura 6: Teste de mesa manuscrito do *Shell-sort*.

3.4 *Bubble Sort*

O *Bubble Sort* é um dos algoritmos de ordenação mais simples, funcionando pela troca repetida de pares de elementos adjacentes que estejam na ordem errada. Apesar de sua simplicidade, é ineficiente para listas grandes.[1]

3.4.1 Funcionamento do *Bubble Sort*

O algoritmo percorre a lista várias vezes, comparando elementos adjacentes e trocando-os quando estão fora de ordem. A cada passagem, o maior elemento é “empurrado” para o final da lista, como bolhas subindo na água.

3.4.2 Passo a Passo do *Bubble Sort*

1. Compare os dois primeiros elementos.
2. Troque-os se estiverem fora de ordem.
3. Passe para o próximo par adjacente e repita.
4. Ao final de cada passagem, o maior elemento estará no fim da lista.
5. Repita até que não haja mais trocas.

3.4.3 Pseudocódigo do *Bubble Sort*

<i>BUBBLE-SORT</i> (A,N)		
PSEUDOCÓDIGO	CUSTO	VEZES
1- FOR i=1 TO n-1	c_1	$(n - 1)$
2- FOR j=1 TO n-i	c_2	$\sum_{i=1}^{n-1} (n - i)$
3- IF A[j]>A[j+1]	c_3	$\sum_{i=1}^{n-1} (n - i)$
4- TROCA A[j] < - > A[j+1]	c_4	$T, T \leq \sum_{i=1}^{n-1} (n - 1)$

Figura 7: Pseudocódigo do *Bubble-Sort* com análise de custo e vezes.

3.4.4 Teste de Mesa do *Bubble Sort*

				1 2 3 4 5
i	j	se $A[j] > A[j+1]$	$A[j] \leftrightarrow A[j+1]$	5 3 8 4 6
1	1	$A[1] > A[2]$ $5 > 3$ ok	$A[1] = 3$ $A[2] = 5$	3 5 8 4 6
	2	$A[2] > A[3]$ $5 > 8$ não		
	3	$A[3] > A[4]$ $8 > 4$ ok	$A[3] \leftrightarrow A[4]$ $A[3] = 4$ e $A[4] = 8$	3 5 4 8 6
	4	$A[4] > A[5]$ $8 > 6$ ok	$A[4] \leftrightarrow A[5]$ $A[4] = 6$ e $A[5] = 8$	3 5 4 6 8
2	1	$A[1] > A[2]$ $3 > 5$ não		
	2	$A[2] > A[3]$ $5 > 4$ ok	$A[2] \leftrightarrow A[3]$ $A[2] = 4$ e $A[3] = 5$	3 4 5 6 8
	3	$A[3] > A[4]$ $5 > 6$ não		ok

Figura 8: Teste de mesa manuscrito do *Bubble-sort*.

3.5 *Merge Sort*

O *Merge Sort* é um algoritmo de ordenação baseado na técnica de divisão e conquista (divide and conquer). Ele divide repetidamente a lista em duas metades, ordena cada metade recursivamente e, em seguida, combina (merge) essas metades de forma ordenada. É um dos algoritmos de ordenação mais eficientes.[1][2]

3.5.1 Funcionamento do *Merge Sort*

O *Merge Sort* funciona dividindo a lista original em sublistas até que cada uma contenha apenas um elemento (o que já significa estar ordenada). Em seguida, as sublistas são combinadas de forma ordenada para formar listas maiores. Esse método garante que o resultado

final seja uma lista totalmente ordenada, independentemente da ordem inicial dos elementos.

3.5.2 Passo a Passo do *Merge Sort*

1. Divida o vetor em duas metades aproximadamente iguais.
2. Aplique o Merge Sort recursivamente em cada metade.
3. Quando restar apenas um elemento em cada sublista, comece o processo de intercalação (merge).
4. Compare os primeiros elementos das duas sublistas e coloque o menor na nova lista.
5. Repita o processo até que todos os elementos tenham sido mesclados em ordem crescente.
6. Retorne a lista final, que estará totalmente ordenada.

3.5.3 Pseudocódigo do *Merge Sort*

<i>MERGE-SORT</i> (A,N)			
PSEUDOCÓDIGO		CUSTO	VEZES
MergeIntercala(A, inicio, meio, fim)			
1-	i = inicio; j = meio + 1; k = 0	c_1	0
2-	WHILE (i<=meio) AND (j<=fim)	c_2	n
3-	IF A[i] <= A[j]	c_3	$(n - 1)$
4-	temp[k++] = A[i++]	c_4	$(n - 1)$
5-	ELSE temp[k+1] = A[j++]	c_5	$(n - 1)$
6-	WHILE (i<=meio)	c_6	n
7-	temp[k++] = A[i++]	c_7	$(n - 1)$
8-	WHILE (j<=fim)	c_8	n
9-	temp[k++] = A[j++]	c_9	$(n - 1)$
10-	FOR i = inicio TO fim	c_{10}	n
11-	A[i] = temp[i-inicio]	c_{11}	$(n - 1)$
MergeRec(A, inicio, fim)			
1-	IF inicio < fim		$O(1)$
2-	meio = (inicio + fim) / 2		$O(1)$
3-	MergeRec(A, inicio, fim)		$T(n/2)$
4-	MergeRec(A, meio + 1, fim)		$T(n/2)$
5-	MergeIntercala(A, inicio, meio, fim)		$O(n)$

Figura 9: Pseudocódigo do *Merge-Sort* com análise de custo e vezes.

3.5.4 Teste de Mesa do Merge Sort

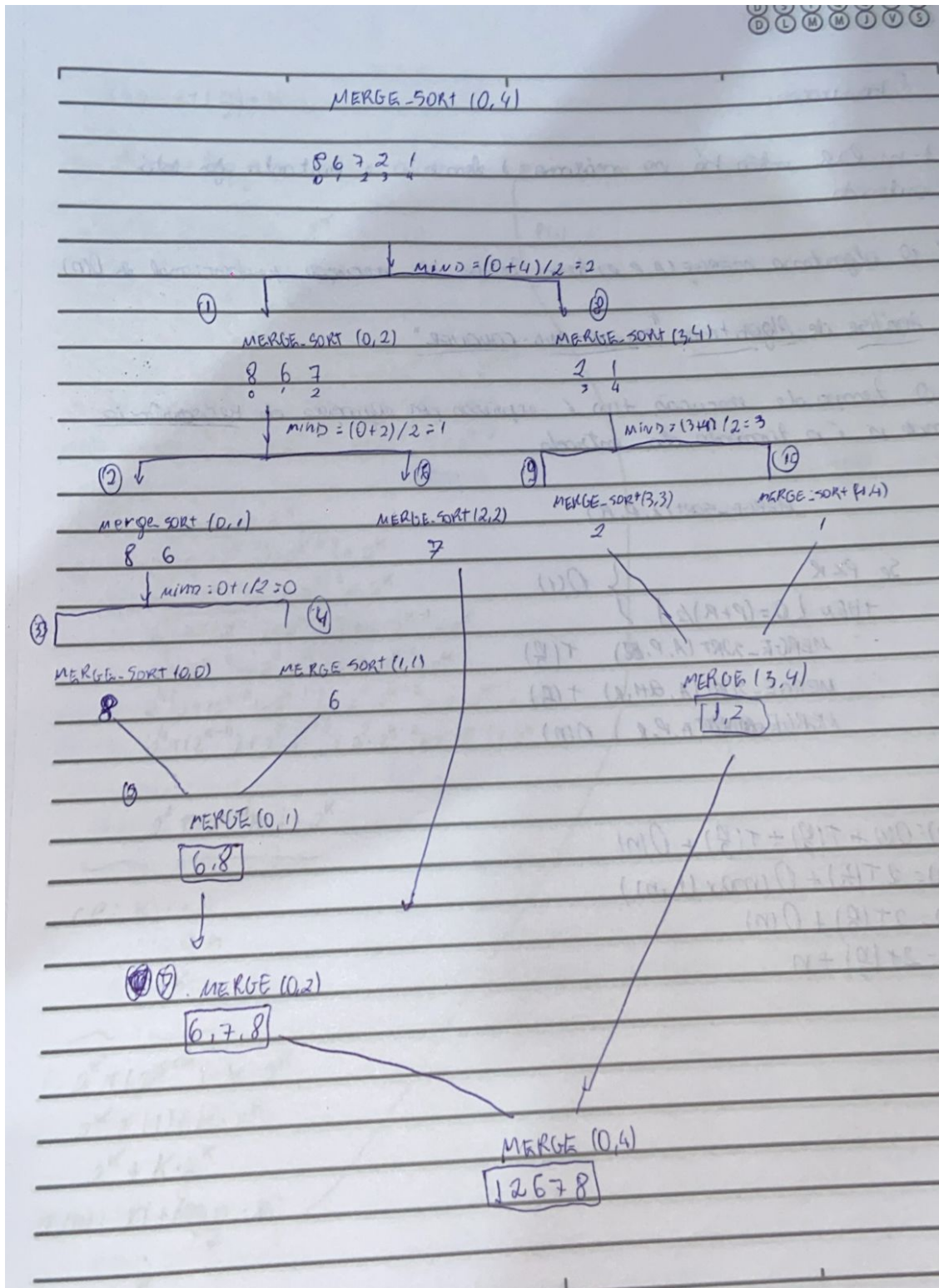


Figura 10: Teste de mesa manuscrito do Merge-sort.

3.6 *Quick Sort*

O *Quick Sort* é um algoritmo de ordenação baseado na técnica de divisão e conquista, descrito originalmente por Tony Hoare em 1962. Ele opera escolhendo um elemento da lista, chamado pivô, e reorganizando os demais elementos em duas partes: à esquerda ficam os elementos menores ou iguais ao pivô, à direita ficam os elementos maiores que o pivô.

Após esse processo, chamado de **particionamento**, o pivô já está na sua posição final ordenada, o que dispensa qualquer etapa de **combinação**, diferentemente do *Merge Sort*. Em seguida, o algoritmo aplica o mesmo procedimento recursivamente às sublistas da esquerda e da direita, até que toda a estrutura esteja ordenada.

O *Quick Sort* é amplamente utilizado na prática devido ao seu excelente desempenho médio, que atinge $O(n \log n)$ quando a escolha do pivô é adequada. Ainda assim, seu pior caso pode chegar a $O(n^2)$, especialmente quando o pivô escolhido é sempre o maior ou o menor elemento da lista. Versões modernas usam estratégias como pivô aleatório ou mediana de três para evitar esse cenário.[1][2]

3.6.1 Funcionamento do *Quick Sort*

O *Quick Sort* escolhe um pivô e reorganiza a lista de forma que todos os elementos menores que o pivô fiquem à sua esquerda, e todos os maiores, à sua direita. Esse processo é chamado de particionamento. Após o particionamento, o pivô fica em sua posição final na lista ordenada. O algoritmo é então aplicado recursivamente às duas partes restantes da lista até que todas estejam ordenadas. O desempenho do *Quick Sort* depende principalmente de como o pivô é escolhido. Em média, o algoritmo é muito eficiente, mas no pior caso (quando o pivô é sempre o menor ou o maior elemento), pode atingir $O(n^2)$.

3.6.2 Passo a Passo do *Quick Sort*

1. Escolha um elemento da lista para ser o pivô.
2. Particione a lista em duas partes: elementos menores que o pivô e elementos maiores que o pivô.
3. Coloque o pivô em sua posição definitiva dentro da lista.

4. Recursivamente aplique o quick à sublista da esquerda (elementos menores).
5. Recursivamente aplique o quick à sublista da direita (elementos maiores).
6. O processo termina quando as sublistas têm tamanho igual a 0 ou 1.

3.6.3 Pseudocódigo do *Quick Sort*

<i>QUICK-SORT(A,N)</i>			
PSEUDOCÓDIGO		CUSTO	VEZES
QuickSort(A, inicio, fim, tipoPivo)			
1-	IF inicio < fim	c_1	1
2-	indicePivo = Particiona(A, inicio, fim, tipoPivo)	c_2	$T(n)$
3-	QuickSort(A, inicio, indicePivo-1, tipoPivo)	c_3	$T(k)$
4-	QuickSort(A, indicePivo+1, fim, tipoPivo)	c_4	$T(n - k - 1)$
Particiona(A, inicio, fim)			
1-	Seleciona pivo conforme tipoPivo	c_5	1
2-	pivoValor = A[pivo]	c_6	1
3-	Trocar A[pivo] com A[fim]	c_7	1
4-	i = inicio - 1	c_8	1
5-	FOR j = inicio TO fim -1	c_9	$(n - 1)$
6-	IF A[j] <= pivoValor	c_{10}	$(n - 1)$
7-	i = i + 1	c_{11}	$(n - 1)$
8-	Trocar A[i] com A[j]	c_{12}	$(n - 1)$
9-	Trocar A[i+1] com A[fim]	c_{13}	1
10-	return i+1	c_{14}	1

Figura 11: Pseudocódigo do *Quick-Sort* com análise de custo e vezes.

3.6.4 Teste de Mesa do *Quick Sort*

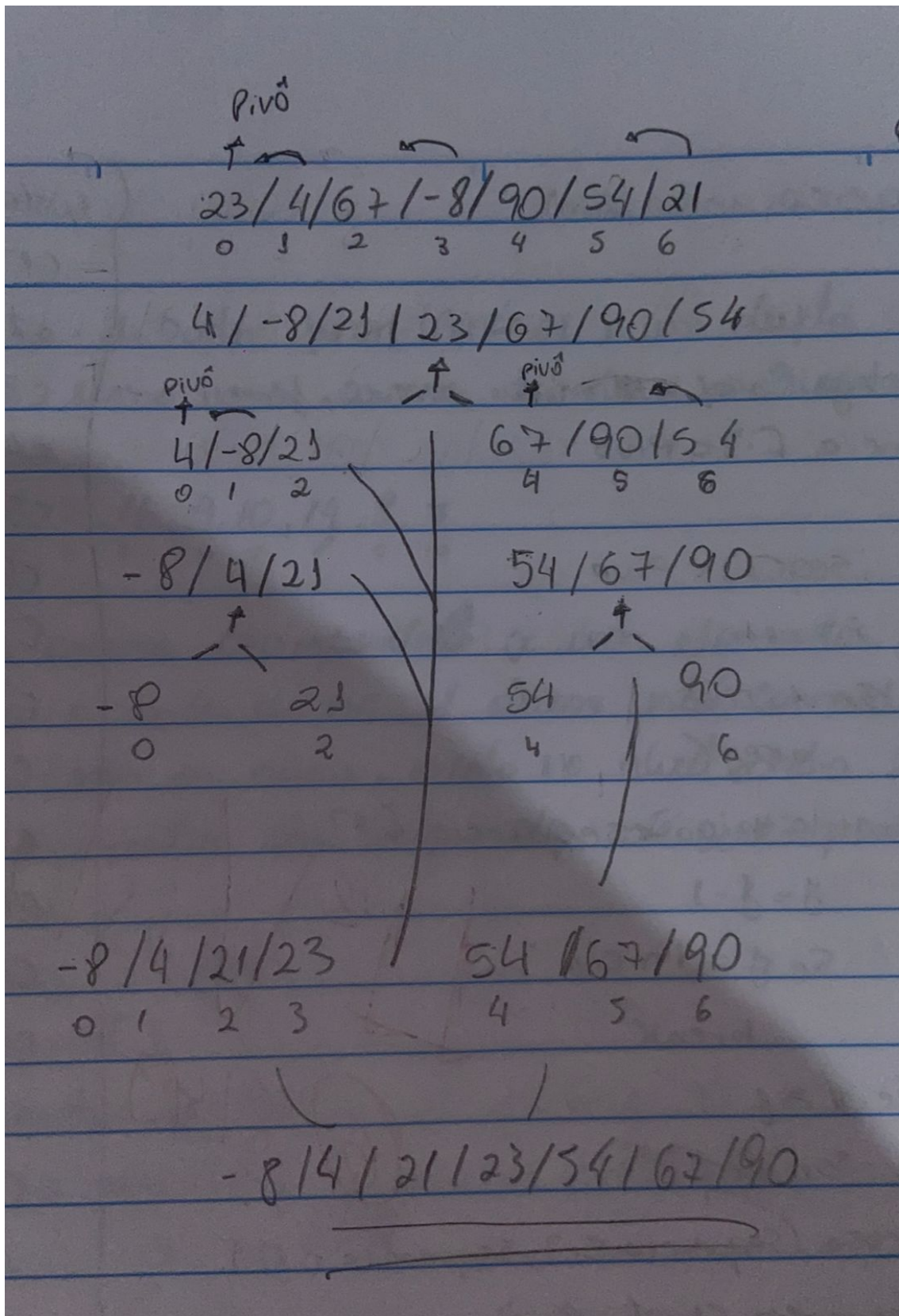


Figura 12: Teste de mesa manuscrito do *Quick-sort*, tendo o primeiro elemento como pivô.

3.7 *Heap Sort*

O *Heap Sort* é um algoritmo de ordenação baseado na estrutura de dados conhecida como **heap**, um tipo especial de árvore binária completa que atende à propriedade de *heap*. No contexto deste trabalho, utiliza-se especificamente um **HEAP MÍNIMO**, no qual cada nó possui valor menor ou igual ao de seus filhos, fazendo com que o menor elemento do conjunto esteja sempre localizado na raiz da estrutura.

O procedimento geral do *Heap Sort* consiste em transformar o vetor de entrada em um **heap mínimo** por meio da operação *BUILD – MIN – HEAP*, e em seguida realizar sucessivas remoções do **menor elemento** por meio de *EXTRACT – MIN*, armazenando cada elemento retirado em ordem crescente. Como o *heap* mínimo sempre fornece o menor elemento disponível, a lista resultante encontra-se ordenada ao final. Diferente de algoritmos como o *Merge Sort*, o *Heap Sort* não realiza combinações, sua ordenação decorre exclusivamente da manutenção da propriedade de *heap* a cada remoção.[1][2][3]

3.7.1 Funcionamento do *Heap Sort (Min-heap)*

O *Heap Sort* com **heap mínimo** inicia convertendo o vetor original em um *heap* válido por meio da função *BUILD-MIN-HEAP*, que reorganiza os elementos de modo a satisfazer a propriedade de *heap* mínimo. Esse processo percorre os nós internos do vetor de forma decrescente, aplicando a rotina *MIN-HEAPIFY* em cada posição, garantindo que cada subárvore obedeça à ordem *Min-Heap*.]

Após a construção da estrutura, o algoritmo passa a extrair repetidamente o menor elemento, sempre localizado na raiz. Cada chamada de *EXTRACT-MIN* remove a raiz, substitui-a pelo último elemento do *heap* e aplica *MIN-HEAPIFY* na posição inicial para restaurar a propriedade do *heap*. Esse processo continua até que todos os elementos tenham sido removidos e colocados na sequência resultante. Assim, a ordenação final emerge naturalmente da retirada dos menores elementos em ordem crescente.

3.7.2 Passo a Passo do *Heap Sort*

Considere o vetor inicial: $V = [7, 3, 9, 1, 4]$

1. Construção do *Heap Mínimo (BUILD-MIN-HEAP)*

O vetor é reorganizado para obedecer à propriedade de *min-heap*: [1,3,9,7,4]

2. Extração do menor elemento (*EXTRACT-MIN*)

Menor elemento: 1

Substitui-se a raiz pelo último elemento

Aplica-se *MIN-HEAPFY*

Resultado parcial: [3,4,9,7] – > Elemento ordenado: 1

3. Nova extração

Menor agora é 3. Resultado: [4,7,9] – > Elementos ordenados: 1,3

4. Repetições até o heap ficar vazio

Ordem final crescente: [1,3,4,7,9]

3.7.3 Pseudocódigo do *Heap Sort (Min-Heap)*

Tabela 1: Pseudocódigo do *Heap-Sort (Min-Heap)* com análise de custo e vezes.

<i>HEAP-SORT(A,N)</i>			
PSEUDOCÓDIGO		CUSTO	VEZES
HeapSort (A,N)			
1-	buildMinHeap(A,N)	c_1	1
2-	para i = 1 até N	c_2	n
3-	B[i] ← extractMin(A)	c_3	n
4-	retornar B	c_4	1
buildMinHeap (A,N)			
5-	heapSize ← N	c_5	1
6-	para i = $\lfloor N/2 \rfloor$ até 1	c_6	$n/2$

<i>HEAP-SORT</i> (A,N) — continuação			
	PSEUDOCÓDIGO	CUSTO	VEZES
7-	minHeapify(A,i)	c_7	$n/2$
	minHeapfy (A,i)		
8-	menor $\leftarrow$ i	c_8	1
9-	esq $\leftarrow$ 2*i	c_9	1
10-	dir $\leftarrow$ 2*i+1	c_{10}	1
11-	se (esq $\leq$ heapSize) e (A[esq] < A[menor])	c_{11}	1
12-	menor $\leftarrow$ esq	c_{12}	1
13-	se (dir $\leq$ heapSize) e (A[dir] < A[menor])	c_{13}	1
14-	menor $\leftarrow$ dir	c_{14}	1
15-	se (menor $\neq$ i)	c_{15}	1
16-	trocar A[i] $\longleftrightarrow$ A[menor]	c_{16}	$O(1)$
17-	minHeapify(A, menor)	c_{17}	$O(\log n)$
	extractMin(A)		
18-	min $\leftarrow$ A[1]	c_{18}	n
19-	A[1] $\leftarrow$ A[heapSize]	c_{19}	n
20-	heapSize $\leftarrow$ heapSize - 1	c_{20}	n
21-	minHeapfy(A,1)	c_{21}	$O(\log n)$
22-	retornar min	c_{22}	n

3.7.4 Teste de Mesa do *Heap Sort* (*Min-Heap*)

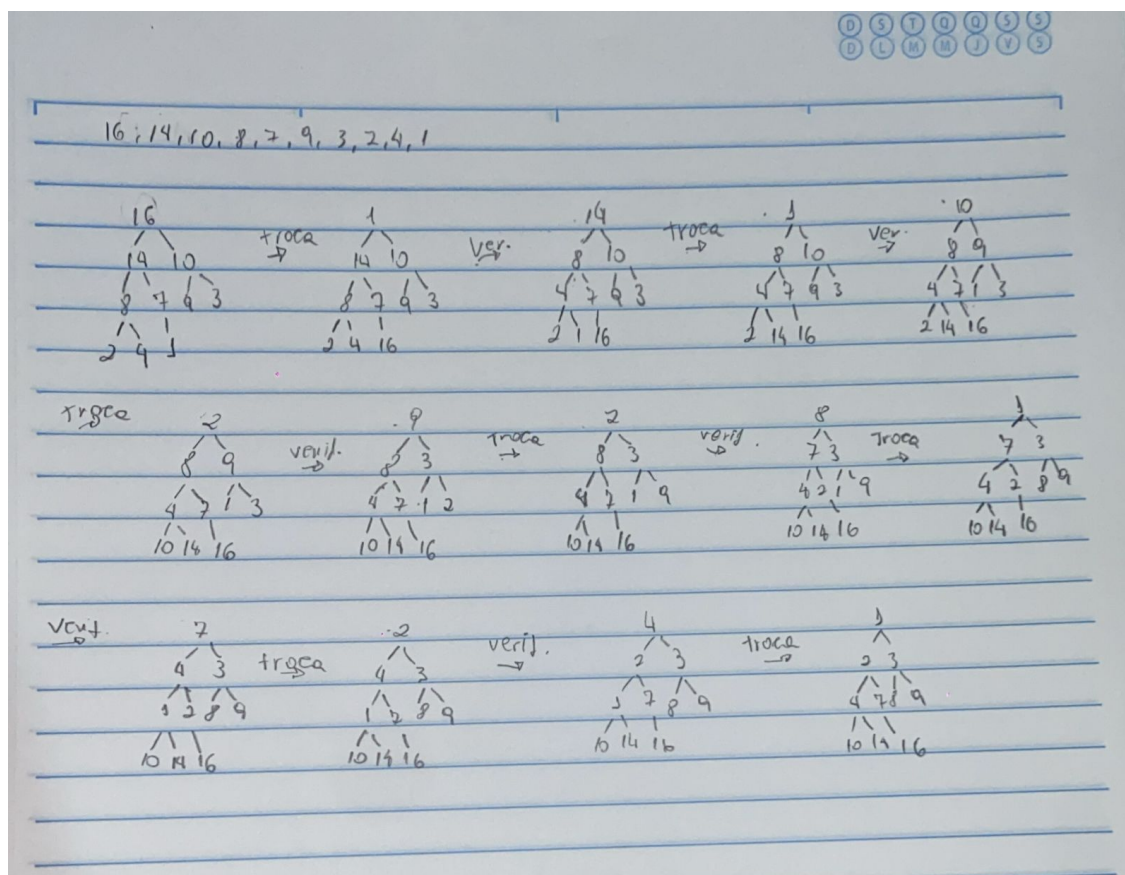


Figura 13: Teste de mesa manuscrito do *Heap-sort* (MIn-Heap).

4 Análise de Complexidade

4.1 Insertion Sort

No *Insertion Sort*, a análise se concentra em quantas comparações e deslocamentos são feitos enquanto cada elemento “encontra seu lugar” na parte já ordenada do vetor. O custo depende bastante de como os elementos chegam: se já estiverem quase na ordem certa, o trabalho é tranquilo; se estiverem bagunçados, cada elemento precisa percorrer a lista inteira.

4.1.1 Melhor Caso - Insertion Sort

No **melhor caso**, quando o vetor já está ordenado, cada elemento é comparado apenas uma vez e não há deslocamentos. Nesse cenário, a complexidade de tempo é $T(n)=A'n+B$, e a notação Big O, temos $O(n)$.

Handwritten derivation of the time complexity for the best case of Insertion Sort:

calcula do melhor caso

arranjo submodo.

Forma Geral

$$T(n) = C_1 \cdot n + C_2(n-1) + C_3(n-1) + C_4(n-1) + C_5(n-1) + C_6(n-1) =$$
$$C_1n + (C_2n - C_2) + (C_3n - C_3) + (C_4n - C_4) + (C_5n - C_5) + (C_6n - C_6)$$
$$(C_1 + C_2 + C_3 + C_4 + C_5 + C_6)n + (-C_2 - C_3 - C_4 - C_5 - C_6)$$

A' B

$$T(n) = A'n + B$$

Figura 14: Análise de complexidade para MELHOR CASO.

4.1.2 Médio Caso - *Insertion Sort*

No **médio caso**, considerando que os elementos estão em ordem aleatória, cada elemento precisa ser comparado e possivelmente deslocado em média metade da sublista à esquerda. A complexidade de tempo aproximada é $T(n) = A''n^2 + B''n + C$, e na notação Big O, temos $O(n^2)$.

Calculo do caso médio

(C3)

$$\sum_{j=2}^n \frac{1}{2} = \sum_{j=2}^n \frac{j-1}{2} = \frac{(n-1)(1+n)}{2} = \frac{n+n^2-1-n}{2} = \frac{n^2-1}{2}$$

(C6)

$$\sum_{j=2}^n \frac{1}{2} - 1 = \sum_{j=2}^n \frac{j-1}{2} - \sum_{j=2}^n 1 = \left(\frac{n^2-1}{2} \right) - (n-1) = \frac{n^2-1-2(n-1)}{2}$$

$$= \frac{n^2-1-2n+2}{2} = \frac{n^2-2n+1}{2}$$

Forma Geral

$$C_1 \cdot n + C_2 \cdot (n-1) + C_3 \cdot (n-1) + C_4 \cdot (n-1) + C_5 \cdot \left(\frac{n^2-1}{2} \right) + C_6 \cdot \left(\frac{n^2-2n+1}{2} \right) + C_7 \cdot \left(\frac{n^2-2n+1}{2} \right) + C_8 \cdot (n-1)$$

$$= \left(C_1 \cdot n + C_2 n - C_2 + C_3 n - C_3 + C_4 n - C_4 + C_5 \frac{n^2-1}{2} - C_5 + C_6 \frac{n^2-2n+1}{2} - C_6 \frac{2n}{2} + C_6 \frac{1}{2} + C_7 \frac{n^2-2n}{2} + C_7 \frac{2n}{2} + C_7 \frac{1}{2} + C_8 n - C_8 \right)$$

$$= \underbrace{(C_5 + C_6 + C_7)}_{A''} \frac{n^2}{2} + \underbrace{(C_1 + C_2 + C_3 + C_4 - \frac{C_6}{2} + C_7 + C_8)}_{B''} n + \underbrace{(-C_2 - C_3 - C_4 - C_5 + \frac{C_6}{2} + C_7 - C_8)}_{C''}$$

$$A'' n^2 + B'' n + C''$$

Figura 15: Análise de complexidade para MÉDIO CASO.

4.1.3 Pior Caso - *Insertion Sort*

No **pior caso**, quando o vetor está ordenado em ordem decrescente, cada elemento percorre toda a sublista à esquerda antes de ser inserido, gerando a máxima quantidade de com-

parações e movimentos. A complexidade de tempo é $T(n) = A'n^2 + B'n + C$, e na notação Big O, temos $O(n^2)$.

cálculo do pior caso

(5) $\sum_{j=2}^n j = \sum_{j=2}^n \frac{(n-1)(2+n)}{2} = \frac{2n^2 - 2 + n^2 - n}{2} = \frac{n^2 + 2n - 2}{2}$ ou $\frac{n^2 + n - 1}{2}$

(6) $\sum_{j=2}^n (j-1) = \sum_{j=2}^n j - \sum_{j=2}^n 1 = \left[\frac{n^2 + n - 1}{2} \right] - [n - 1] = \frac{n^2 - n}{2}$

Forma Geral

$$T(n) = C_1 \cdot n + C_2(n-1) + C_3(n-1) + C_4(n-1) + C_5 \left(\frac{n^2 + n - 1}{2} \right) + C_6 \left(\frac{n^2 - n}{2} \right) + C_7 \left(\frac{n^2 - n}{2} \right) + C_8(n-1)$$

$$= C_1 \cdot n + C_2 n - C_2 + C_3 n - C_3 + C_4 n - C_4 + \frac{C_5 n^2}{2} + \frac{C_5 n}{2} - \frac{C_5}{2} + \frac{C_6 n^2}{2} - \frac{C_6 n}{2} - \frac{C_6}{2} + \frac{C_7 n^2}{2} - \frac{C_7 n}{2} - \frac{C_7}{2} - C_8 n + C_8$$

$$= \frac{C_5 n^2}{2} + \frac{C_6 n^2}{2} + \frac{C_7 n^2}{2} - \frac{C_2}{2} + \frac{C_3}{2} + \frac{C_4}{2} + \frac{C_5 n}{2} - \frac{C_6 n}{2} - \frac{C_7 n}{2} - C_8 n + C_8 - \frac{C_5}{2} - \frac{C_6}{2} - \frac{C_7}{2}$$

$$= \left(\frac{C_5 + C_6 + C_7}{2} \right) n^2 + \left(\frac{C_5}{2} - \frac{C_6}{2} - \frac{C_7}{2} - C_8 \right) n + \left(\frac{C_3}{2} - \frac{C_2}{2} - \frac{C_5}{2} - \frac{C_6}{2} - \frac{C_7}{2} + C_8 \right)$$

$$= \alpha n^2 + \beta n + \epsilon$$

Figura 16: Análise de complexidade para PIOR CASO.

4.2 Selection Sort

Para analisar a complexidade do *Selection Sort*, observamos quantas vezes o algoritmo percorre o vetor para escolher o menor elemento a cada rodada. Como ele sempre passa pelo vetor inteiro, o número de operações é mais previsível e não depende de como os elementos estão inicialmente.

4.2.1 Melhor Caso - Selection Sort

No melhor caso, quando o vetor já está ordenado em ordem crescente, o *Selection Sort* ainda precisa percorrer toda a lista para encontrar o menor elemento em cada iteração. Por-

tanto, mesmo sem trocas efetivas, o algoritmo realiza a mesma quantidade de comparações. A complexidade de tempo é $T(n) = An^2 + Bn + C$ e na notação Big O, temos $O(n^2)$.

$$\sum_{i=1}^{n-1} (n-i) = 1 + 2 + \dots + (n-1) = \frac{n(1+(n-1))}{2} = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}$$

P.A

Melhor caso (o if nunca é verdadeiro, as linhas não executam)

$$T(n) = c_1(n-1) + c_2(n-1) + c_3\left(\frac{n^2}{2}\right) + c_4\left(\frac{n^2}{2}\right) + c_6(n-1) =$$

$$(c_1n) - c_1 + (c_2n) - c_2 + \left(\frac{c_3n^2}{2}\right) + \left(\frac{c_4n^2}{2}\right) + (c_6n) - c_6 =$$

$$\underbrace{\left(\frac{c_3}{2} + \frac{c_4}{2}\right)}_A n^2 + \underbrace{(c_1 + c_2 + c_6)}_B n + \underbrace{(-c_1 - c_2 - c_6)}_C$$

$$T(n) = A'n^2 + B'n + C'$$

Figura 17: Análise de complexidade para MELHOR CASO.

4.2.2 Médio Caso - *Selection Sort*

No caso médio, com elementos em ordem aleatória, o algoritmo continua realizando a busca pelo menor elemento em cada iteração do laço externo. Cada elemento é comparado com todos os elementos restantes da sublista, gerando um número de comparações aproximadamente igual ao do pior caso. A complexidade de tempo também é $T(n) = An^2 + Bn + C$, ou seja, $O(n^2)$.

CASO MÉDIO (C3 é verdade retradi das vezes, C5/2)

$$\frac{n^2}{2} = \frac{n^2}{2} \cdot \frac{1}{2} = \frac{n^2}{4} \rightarrow C5$$

$$T(n) = C1(n-1) + C2(n-1) + C3\left(\frac{n^2}{2}\right) + C4\left(\frac{n^2}{2}\right) + C5\left(\frac{n^2}{4}\right) + C6(n-1) =$$

$$(C3n)C1 + (C2n)C2 + \left(C3\frac{n^2}{2}\right) + \left(C4\frac{n^2}{2}\right) + \left(C5\frac{n^2}{4}\right) + (C6n)C6 =$$

$$\underbrace{\left(\frac{C3}{2} + \frac{C4}{2} + \frac{C5}{4}\right)}_A n^2 + \underbrace{(C1 + C2 + C6)}_B n + \underbrace{(-C1 - C2 - C6)}_C$$

$$T(n) = A''n^2 + B''n + C$$

Figura 18: Análise de complexidade para MÉDIO CASO.

4.2.3 Pior Caso - *Selection Sort*

No pior caso, quando o vetor está ordenado em ordem decrescente, o algoritmo ainda precisa percorrer toda a sublista restante para localizar o menor elemento antes de realizar a troca. Isso gera a máxima quantidade de comparações e um número de movimentos igual ao do melhor caso, já que o Selection Sort realiza no máximo uma troca por iteração do laço externo. A complexidade de tempo é $T(n) = An^2 + Bn + C$, e na notação Big O, temos $O(n^2)$.

Pior CASO (ocorre quando todos estão desordenados)

$$T(n) = C_1(n-1) + C_2(n-1) + C_3\left(\frac{n^2}{2}\right) + C_4\left(\frac{n^2}{2}\right) + C_5\left(\frac{n^2}{2}\right) + C_6(n-1) =$$

$$C_1n - C_1 + C_2n - C_2 + \underbrace{C_3\frac{n^2}{2}} + \underbrace{C_4\frac{n^2}{2}} + \underbrace{C_5\frac{n^2}{2}} + \underbrace{C_6n - C_6} =$$

$$\underbrace{\left(\frac{C_3 + C_4 + C_5}{2}\right)n^2}_A + \underbrace{(C_1 + C_2 + C_6)n}_B + \underbrace{(-C_1 - C_2 - C_6)}_C$$

$$T(n) = A'n^2 + B'n + C$$

Figura 19: Análise de complexidade para PIOR CASO.

4.3 *Shell Sort*

A complexidade do *Shell Sort* é determinada pelos saltos (*gaps*) usados para comparar e mover os elementos. Diferentemente de outros algoritmos, o desempenho depende fortemente da sequência de saltos e da distribuição dos valores no vetor, tornando o número exato de comparações e movimentos difícil de prever. Por esse motivo, **não é possível realizar uma análise manuscrita** detalhada dos casos, sendo a avaliação feita de forma aproximada, considerando as características do algoritmo e o comportamento típico para diferentes tipos de entrada.

4.3.1 Melhor Caso - *Shell Sort*

No melhor caso, quando o vetor já está ordenado, o *Shell Sort* ainda percorre o vetor utilizando os *gaps* definidos pela sequência escolhida. Como não há muitas trocas a serem feitas, o número de comparações é reduzido, resultando em complexidade aproximada de $O(n \log n)$.

4.3.2 Médio Caso - *Shell Sort*

Para um vetor com elementos em ordem aleatória, o número de comparações e movimentos é intermediário, dependendo tanto da distribuição dos valores quanto da sequência de *gaps*. A complexidade média fica entre $O(n^{1.25})$ e $O(n^{1.5})$, sendo mais eficiente que o *Bubble* ou *Selection* na maioria das situações.

4.3.3 Pior Caso - *Shell Sort*

No pior caso, quando o vetor está ordenado em ordem decrescente, cada elemento precisa percorrer os *gaps* até alcançar sua posição correta, gerando o máximo de comparações e movimentos possíveis para a sequência de *gaps* utilizada. Para a sequência clássica de *Shell*, a complexidade chega a $O(n^2)$.

4.4 *Bubble Sort*

No *Bubble Sort*, a análise foca nas comparações e trocas entre elementos vizinhos. O custo muda bastante dependendo da ordem inicial: se o vetor já estiver quase ordenado, o trabalho é bem menor; se estiver invertido, cada elemento precisa percorrer todo o vetor.

4.4.1 Melhor Caso - *Bubble Sort*

No melhor caso, quando o vetor já está ordenado, o *Bubble Sort* clássico (sem otimização) ainda percorre todo o vetor realizando comparações, mas nenhuma troca é necessária, mantendo o número de movimentos menor. A complexidade de tempo continua sendo $T(n) = A'n^2 + B'n + C$, ou seja $O(n^2)$. Se utilizarmos uma versão otimizada, que interrompe o algoritmo ao detectar que não há trocas, o melhor caso passa a ter notação Big O, $O(n)$.

P.A

$$\sum_{i=1}^{n-1} (n-i) = (n-1) + \dots + (n-(n-1)) = 1 + \dots + (n-1) = \frac{(n-1)(1+(n-1))}{2} =$$

$$= \frac{(n-1)(1+n-1)}{2} = \frac{n(n-1)}{2} = \frac{n^2-n}{2} //$$

$$\sum_{i=1}^{n-1} (n-i) = 1 + 2 + \dots + (n-1) = \frac{(n-1)(1+n-1)}{2} = \frac{(n-1)(n)}{2} = \frac{n^2-n}{2} //$$

MELHOR CASO (já ordenado)

$$T(n) = C_1(n-1) + C_2\left(\frac{n^2-n}{2}\right) + C_3\left(\frac{n^2-n}{2}\right) =$$

$$\frac{C_1n - C_1}{1} + \frac{C_2n^2 - C_2n}{2} + \frac{C_3n^2 - C_3n}{2} =$$

$$\underbrace{\left(\frac{C_2 + C_3}{2}\right)}_A n^2 + \underbrace{\left(C_1 - \frac{C_2}{2} - \frac{C_3}{2}\right)}_B n + \underbrace{(-C_1)}_C$$

$$T(n) = A'n^2 + B'n + C$$

Figura 20: Análise de complexidade para MELHOR CASO.

4.4.2 Médio Caso - *Bubble Sort*

Para um vetor em ordem aleatória, o algoritmo realiza comparações e trocas em uma quantidade intermediária entre o melhor e o pior caso. Mesmo assim, o termo dominante permanece quadrático, então a complexidade é $T(n) = A'n^2 + B'n + C$ e tem uma notação Big O, $O(n^2)$.

MÉTODO CASO

$$\frac{\frac{n^2-n}{2}}{2} = \frac{n^2-n}{2} \cdot \frac{1}{2} = \frac{n^2-n}{4}$$

$$T(n) = c_1(n-1) + c_2\left(\frac{n^2-n}{2}\right) + c_3\left(\frac{n^2-n}{2}\right) + c_4\left(\frac{n^2-n}{4}\right) =$$

$$\underbrace{c_1 n - c_1}_{\left(\frac{c_1 n}{2}\right)} + \underbrace{\frac{c_2 n^2}{2}}_{\left(\frac{c_2 n^2}{2}\right)} - \underbrace{\frac{c_2 n}{2}}_{\left(\frac{c_2 n}{2}\right)} + \underbrace{\frac{c_3 n^2}{2}}_{\left(\frac{c_3 n^2}{2}\right)} - \underbrace{\frac{c_3 n}{2}}_{\left(\frac{c_3 n}{2}\right)} + \underbrace{\frac{c_4 n^2}{4}}_{\left(\frac{c_4 n^2}{4}\right)} - \underbrace{\frac{c_4 n}{4}}_{\left(\frac{c_4 n}{4}\right)} =$$

$$\underbrace{\left(\frac{c_2}{2} + \frac{c_3}{2} + \frac{c_4}{4}\right)n^2}_{A} + \underbrace{\left(c_1 + \frac{c_2}{2} - \frac{c_3}{2} - \frac{c_4}{4}\right)n}_{B} + \underbrace{(-c_1)}_C$$

$$T(n) = A'n^2 + B'n + C$$

Figura 21: Análise de complexidade para MÉDIO CASO.

4.4.3 Pior Caso - *Bubble Sort*

No pior caso, quando o vetor está ordenado em ordem decrescente, cada elemento precisa “caminhar” até a sua posição correta, gerando a máxima quantidade de comparações e movimentos. Cada elemento é comparado com todos os demais elementos da sublista, e diversas trocas são realizadas. A complexidade de tempo é $T(n) = A'n^2 + B'n + C$, e na notação Big O, temos $O(n^2)$.

PIOR CASO (TUDO DESORDENADO)

$$T(n) = C_1(n-1) + C_2\left(\frac{n^2-n}{2}\right) + C_3\left(\frac{n^2-n}{2}\right) + C_4\left(\frac{n^2-n}{2}\right) =$$

$$\underbrace{C_1 n - C_1}_{A} + \underbrace{\frac{C_2 n^2 - C_2 n}{2}}_{B} + \underbrace{\frac{C_3 n^2 - C_3 n}{2}}_{B} + \underbrace{\frac{C_4 n^2 - C_4 n}{2}}_{B} =$$

$$\underbrace{\left(\frac{C_2 + C_3 + C_4}{2}\right) n^2}_{A} + \underbrace{\left(\frac{C_1 - C_2 - C_3 - C_4}{2}\right) n}_{B} + \underbrace{(-C_1)}_{C}$$

$$T(n) = An^2 + Bn + C$$

Figura 22: Análise de complexidade para PIOR CASO.

4.5 *Merge Sort*

A análise de complexidade do *Merge Sort* baseia-se no fato de que o algoritmo divide recursivamente o vetor em duas metades até que reste apenas um elemento em cada subvetor. Em seguida, realiza a etapa de intercalação (*merge*), unindo essas partes de forma ordenada. Em cada nível de divisão, todos os elementos do vetor são percorridos durante o processo de fusão.

Ao realizar o cálculo do tempo de execução, utilizando as propriedades do Big O e a fórmula de recorrência do algoritmo, observa-se que o custo da etapa de intercalação é $O(n)$ em cada nível, e o número de níveis da recursão é proporcional a $\log n$. Portanto, a complexidade total do *Merge Sort* é $O(n \log n)$.

fórmula geral:

$$T(n) = C_1 \cdot 0 + C_2(n) + C_3(n-1) + C_4(n-1) + C_5(n-1) + C_6(n) + C_7(n-1) + C_8(n) + C_9(n-1) + C_{10}(n) + C_{11}(n-1)$$

$$C_{11}(n-1) =$$

$$= C_2n + C_3n - C_3 + C_4n - C_4 + C_5n - C_5 + C_6n + C_7n - C_7 + C_8n + C_9n - C_9 + C_{10}n + C_{11}n - C_{11}$$

$$(C_2 + C_3 + C_4 + C_5 + C_6 + C_7 + C_8 + C_9 + C_{10} + C_{11})n + (-C_3 - C_4 - C_5 - C_7 - C_9 - C_{11})$$

$$T(n) = A'n + B$$

$$T(n) = O(n)$$

tempo de execução: $T(n) = O(1) + O(n) + T(\frac{n}{2}) + T(\frac{n}{2}) + O(n)$

$$T(n) = O(\max(1, n)) + 2T(n/2)$$

$$T(n) = 2T(n/2) + n$$

$T(n) = 2T(n/2) + n$
 $n = 2^k; k = \log_2 n; T(2^0) = 1$

$T(2^k) = 2T(2^{k-1}) + 2^k = 2T(2^{k-1}) + 2^k$
 $T(2^{k-1}) = 2T(2^{k-2}) + 2^{k-1} = 2T(2^{k-2}) + 2^{k-1}$
 $T(2^{k-2}) = 2T(2^{k-3}) + 2^{k-2} = 2T(2^{k-3}) + 2^{k-2}$

CP: $k-i=0$
 $i=k$
 $2^k T(2^{k-k}) + 2^k \cdot k$
 $2^k T(2^0) + 2^k \cdot k$
 $2^k \cdot 1 + 2^k \cdot k$

FE: $T(n) = n + n \log_2 n$

$2T(2^{k-1}) + 2^k$
 $2[2T(2^{k-2}) + 2^{k-1}] + 2^k$
 $2^2 T(2^{k-2}) + 2 \cdot 2^{k-1} + 2^k = 2^2 T(2^{k-2}) + 2 \cdot 2^k$
 $2^2 [2T(2^{k-3}) + 2^{k-2}] + 2 \cdot 2^{k-1} + 2^k$
 $2^3 T(2^{k-3}) + 2^2 \cdot 2^{k-2} + 2 \cdot 2^{k-1} + 2^k = 2^3 T(2^{k-3}) + 3 \cdot 2^k$
 $\vdots$
 $2^k T(2^{k-k}) + 2^k \cdot k$

$T(n) = O(n \log n)$

Figura 23: Análise de complexidade do Merge Sort.

4.5.1 Melhor Caso - Merge Sort

No melhor caso, o vetor já se encontra ordenado. Ainda assim, o algoritmo executa todas as divisões recursivas e as etapas de intercalação, percorrendo todos os elementos em cada nível de recursão. Dessa forma, o número total de operações não é reduzido de forma significativa, mantendo a mesma ordem de crescimento. A complexidade de tempo é $T(n) = An \log n + Bn$, ou seja, $O(n \log n)$.

4.5.2 Médio Caso - *Merge Sort*

No caso médio, com elementos em ordem aleatória, o comportamento do algoritmo é semelhante ao do melhor caso, pois o processo de divisão e fusão é realizado da mesma forma, independentemente da distribuição dos dados. Assim, o custo total continua proporcional a $n \log n$. A complexidade de tempo é $T(n) = An \log n + Bn$, ou seja, $O(n \log n)$.

4.5.3 Pior Caso - *Merge Sort*

No pior caso, quando o vetor está em ordem inversa, o algoritmo ainda realiza todas as divisões e fusões completas. No entanto, como o procedimento de intercalação percorre todos os elementos em cada nível, o número total de operações segue o mesmo padrão dos demais casos. Assim, a complexidade de tempo permanece $T(n) = An \log n + Bn$, ou seja, $O(n \log n)$.

4.6 *Quick Sort*

No *Quick Sort*, a análise de complexidade é baseada na forma como o vetor é particionado a cada chamada recursiva. Dependendo da escolha do pivô e do estado inicial do vetor, o desempenho pode variar significativamente. Quando o pivô divide bem o vetor, o algoritmo se torna bastante eficiente. Porém, divisões ruins levam ao seu pior comportamento.

4.6.1 Melhor Caso - *Quick Sort*

No melhor caso, o pivô sempre divide o vetor exatamente ao meio (ou de forma muito próxima). Assim, cada chamada recursiva trabalha com duas metades de tamanho igual. O particionamento sempre percorre o vetor inteiro uma vez, resultando em um custo $O(n)$ por nível da recursão. A recorrência do melhor caso é $T(n) = 2T(n/2) + n$, aplicando o Teorema Mestre, temos que em notação Big O o comportamento do *Quick Sort* é $O(n \log n)$.

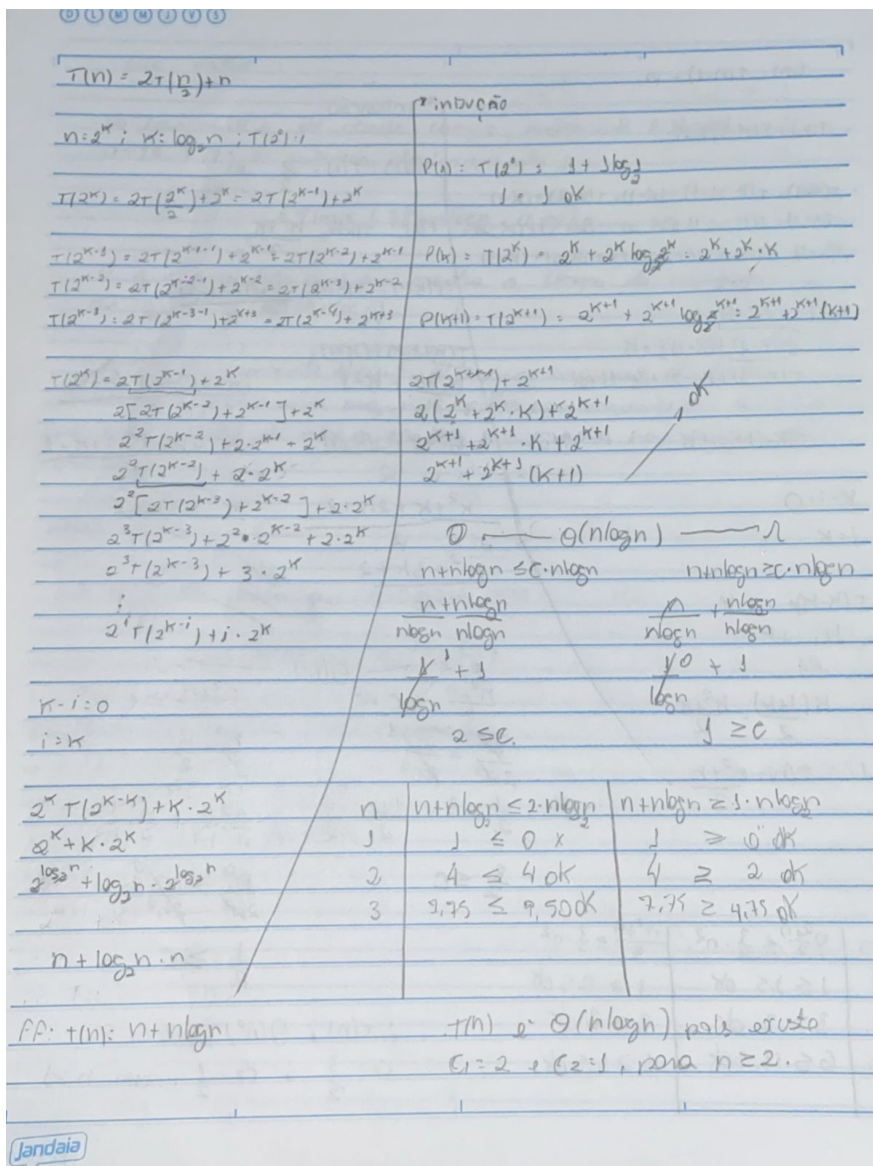


Figura 24: Análise de complexidade para MELHOR CASO.

4.6.2 Médio Caso - Quick Sort

No caso médio, de acordo com a análise clássica de Flajolet e Sedgewick (1996), o número esperado de comparações no *Quick Sort* é dado por: $T(n) \approx 1,386n \log n - 0,846n$.

Esse resultado mostra que, em médio, o *Quick Sort* realiza aproximadamente $1,386 n \log n$ comparações, menos um termo linear de $0,846n$. A constante $1,386 (\approx 2/\ln 2)$ vem da análise probabilística refinada desses autores, e reforça a eficiência prática do algoritmo. Por isso, mesmo considerando as variações probabilísticas na divisão (pivôs não perfeitos),

o algoritmo permanece $O(n \log n)$ no caso médio, com coeficiente constante informado por Flajolet e Sedgewick.[4]

4.6.3 Pior Caso - *Quick Sort*

O pior caso ocorre quando o pivô escolhido gera a pior divisão possível do vetor, como por exemplo:

- O pivô sempre é o primeiro ou último elemento;
- o vetor já se encontra ordenado;
- vetor ordenado ao contrário;
- ou quando todos os elementos são iguais.

Nessas situações, a divisão ocorre na forma: $T(n) = T(n-1) + n$, também aplicando Teorema Mestre temos que a notação Big O do algoritmo é $O(n^2)$.

00100000
00000000

$$T(n) = T(n-1) + n$$

$$T(k) = T(k-1) + k$$

$$T(k-1) = T(k-1-1) + (k-1) = T(k-2) + (k-1)$$

$$T(k-2) = T(k-2-1) + (k-2) = T(k-3) + (k-2)$$

$$T(k-3) = T(k-3-1) + (k-3) = T(k-4) + (k-3)$$

$$T(k) = T(k-1) + k$$

$$\quad [T(k-2) + (k-1)] + k$$

$$\quad + (k-3) + (k-2) + (k-1) + k$$

$$\quad \vdots$$

$$\quad + (k-i) + \dots + k$$

$$k-i=0$$

$$i=k$$

$$T(k-k) + \dots + k$$

$$1 + \dots + k$$

$$PA$$

$$\frac{k(1+k)}{2} = \frac{k^2+k}{2}$$

$$FF: T(n) = \frac{n^2+n}{2}$$

indução

$$P(1): T(1) = \frac{2}{2} \quad OK$$

$$P(k): T(k) = \frac{k^2+k}{2}$$

$$P(k+1): T(k+1) = \frac{(k+1)^2 + (k+1)}{2}$$

$$\frac{T(k+1) + (k+1)}{2} = \frac{\frac{k^2+k}{2} + k + 1}{2} = \frac{(k+1)(k+1) + (k+1)}{2} = \frac{(k+1)(k+1+1)}{2} = \frac{(k+1)(k+2)}{2}$$

$$\frac{k^2+k+2k+2}{2} = \frac{k^2+3k+2}{2}$$

$$\frac{k^2+k+2k+2}{2} = \frac{k^2+3k+2}{2} \quad OK$$

$$0 \leq \frac{n^2+n}{2} \leq c \cdot n^2 \quad \theta(n^2)$$

$$\frac{n^2+n}{2} \geq c \cdot n^2$$

$$\frac{n^2}{2n^2} + \frac{n}{2n^2} = \frac{1}{2} + \frac{1}{2n}$$

$$\frac{1}{2} + \frac{1}{2} = \frac{1+1}{2} = \frac{2}{2} = 1$$

$$\frac{3}{2} \leq c$$

$$\frac{1}{2} \geq c$$

n	$\frac{n^2+n}{2} \leq \frac{3}{2} \cdot n^2$	$\frac{n^2+n}{2} \geq \frac{1}{2} \cdot n^2$
1	$1 \leq 1,5 \quad OK$	$1 \geq 0,5 \quad OK$
2	$3 \leq 3 \quad OK$	$3 \geq 1 \quad OK$
3	$6 \leq 4,5 \quad OK$	$6 \geq 2,25 \quad OK$

$\therefore T(n) = \theta(n^2)$ pois existe

$c_1 = \frac{3}{2} \quad c_2 = \frac{1}{2} \quad \text{para } n \geq 1$

Figura 25: Análise de complexidade para PIOR CASO.

4.7 Heap Sort

O *Heap Sort*, apresenta comportamento assintótico muito estável, independentemente da configuração inicial do vetor. Isso ocorre porque as operações fundamentais do algoritmo, construção do *heap* e extração dos elementos, dependem principalmente da altura da estrutura de *heap* (uma árvore binária quase completa), e não da ordem inicial dos dados. O *Heap Sort* é um dos poucos algoritmos cuja complexidade é uniforme nos três cenários, graças à previsibilidade estrutural do *heap*.

Apesar disso:

- Não é estável, ao contrário de *Merge Sort*;
- Tem mais trocas e acessos à memória do que *Quick Sort* (com pivô otimizado), podendo ser mais lento na prática;
- É ótimo para aplicações que exigem previsibilidade de desempenho.

4.7.1 Melhor Caso - *Heap Sort (Min-Heap)*

No melhor caso, o vetor está em uma ordenação que facilita a reorganização da estrutura de heap durante a fase de build. Entretanto, mesmo que os elementos já satisfaçam parte das propriedades do heap, o algoritmo ainda executa *BUILD-MIN-HEAP* em $O(n)$ e *EXTRACT-MIN* em $O(\log n)$. Como são realizadas N extrações, o tempo total é: $T(n) = O(n) + n \cdot O(\log n) = O(n \log n)$.

4.7.2 Médio Caso - *Heap Sort (Min-Heap)*

No caso médio, assume-se que o vetor está em uma ordem aleatória. Mesmo assim, a análise permanece praticamente idêntica ao melhor caso, *BUILD-MIN-HEAP* sempre percorre metade do vetor com custo amortizado linear, cada *EXTRACT-MIN* sempre realiza uma decida no *heap* de altura $\log n$. Diferente do Quick Sort, o Heap Sort não sofre variações significativas no caso médio, pois sua estrutura não depende da ordem dos dados, portando: $T(n) = O(n) + n \cdot O(\log n) = O(n \log n)$.

4.7.3 Pior Caso - *Heap Sort (Min-Heap)*

Mesmo no pior caso possível, o vetor em ordem que causa o maior número de reorganizações durante o heapify, o algoritmo mantém exatamente os mesmos custos:

- A construção continua $O(n)$;
- Cada remoção continua $O(\log n)$;
- O número de chamadas para *heapify* é sempre n .

Não existe um cenário em que o *Heap Sort* tenha degradação como ocorre no *Quick Sort* (que pode cair para $O(n^2)$ com má escolha de pivô). Logo, no pior caso ele recebe **$O(n \log n)$** .

5 Tabela e Gráfico

5.1 *Insertion Sort*

Tabela 2: Tempos de execução em segundos para diferentes entradas (*Insertion-Sort*)

	TAMANHO DA ENTRADA					
TIPO	10	100	1000	10000	100000	1000000
Crescente	0.000004	0.000005	0.000011	0.000041	0.000266	0.002666
Decrescente	0.000002	0.000017	0.001514	0.106543	9.308213	1109.132003
Randômico	0.000002	0.000026	0.000544	0.046389	4.776612	567.916833

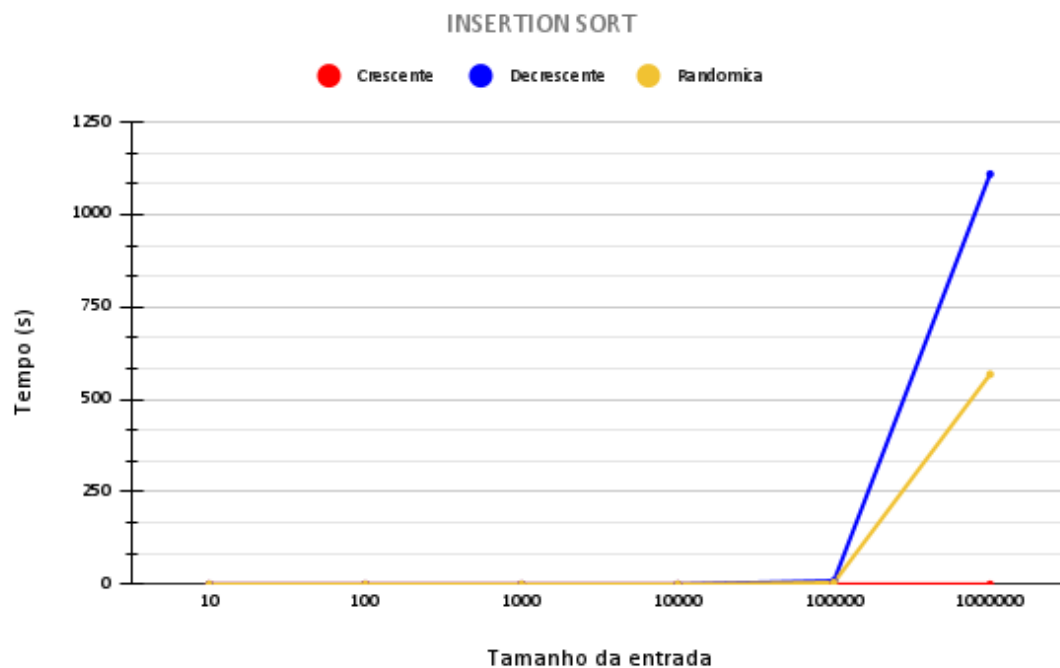


Figura 26: Gráfico de tempo de execução do *Insertion-sort*.

Como pode ser observado no gráfico (Figura 26) e na tabela 2 para entradas de até 1000 elementos, o tempo de execução do algoritmo apresenta variações pouco significativas. No entanto, para entradas maiores que esse valor, nota-se um aumento considerável no

tempo, indicando que a complexidade do algoritmo passa a impactar fortemente a performance. Assim tendo como: **melhor caso(crescente)**, **médio caso(randômica)** e **pior caso(decrescente)**.

5.2 Selection Sort

Tabela 3: Tempos de execução em segundos para diferentes entradas (*Selection-Sort*)

	TAMANHO DA ENTRADA					
TIPO	10	100	1000	10000	100000	1000000
Crescente	0.000002	0.000015	0.001050	0.098688	9.838451	996.422985
Decrescente	0.000002	0.000026	0.002159	0.191703	19.514748	1992.846278
Randômico	0.000002	0.000029	0.003441	0.275458	29.332271	2922.656336

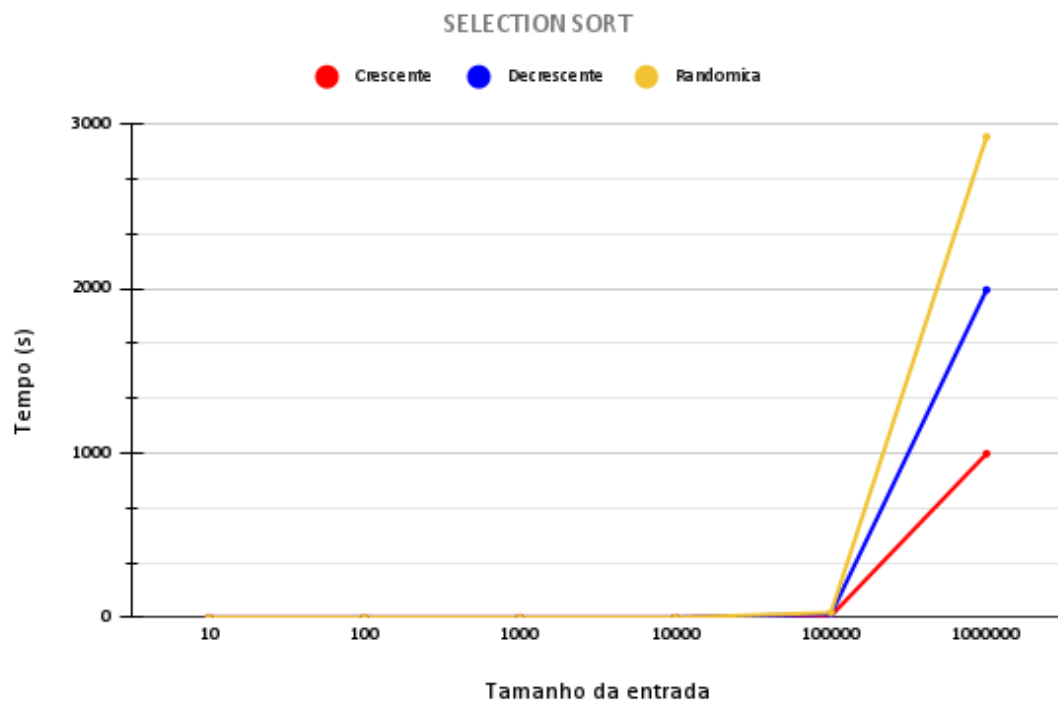


Figura 27: Gráfico de tempo de execução do *Selection-sort*.

A partir da análise do gráfico (Figura 27) e da tabela 3, percebe-se que, para vetores de até 1000 elementos, o tempo de execução do algoritmo sofre variações mínimas. Contudo, quando o tamanho da entrada ultrapassa esse limite, observa-se um crescimento expressivo no tempo gasto, evidenciando o impacto da complexidade na eficiência do algoritmo. Nesse contexto, consideram-se: **melhor caso (entrada crescente)**, **caso médio (entrada decrescente)** e **pior caso (entrada randômica)**.

5.3 *Shell Sort*

Tabela 4: Tempos de execução em segundos para diferentes entradas (*Shell-Sort*)

	TAMANHO DA ENTRADA					
TIPO	10	100	1000	10000	100000	1000000
Crescente	0.000003	0.000003	0.000023	0.000190	0.002412	0.029447
Decrescente	0.000003	0.000006	0.000027	0.000348	0.004011	0.047476
Randômico	0.000002	0.000014	0.000079	0.001203	0.017492	0.265013

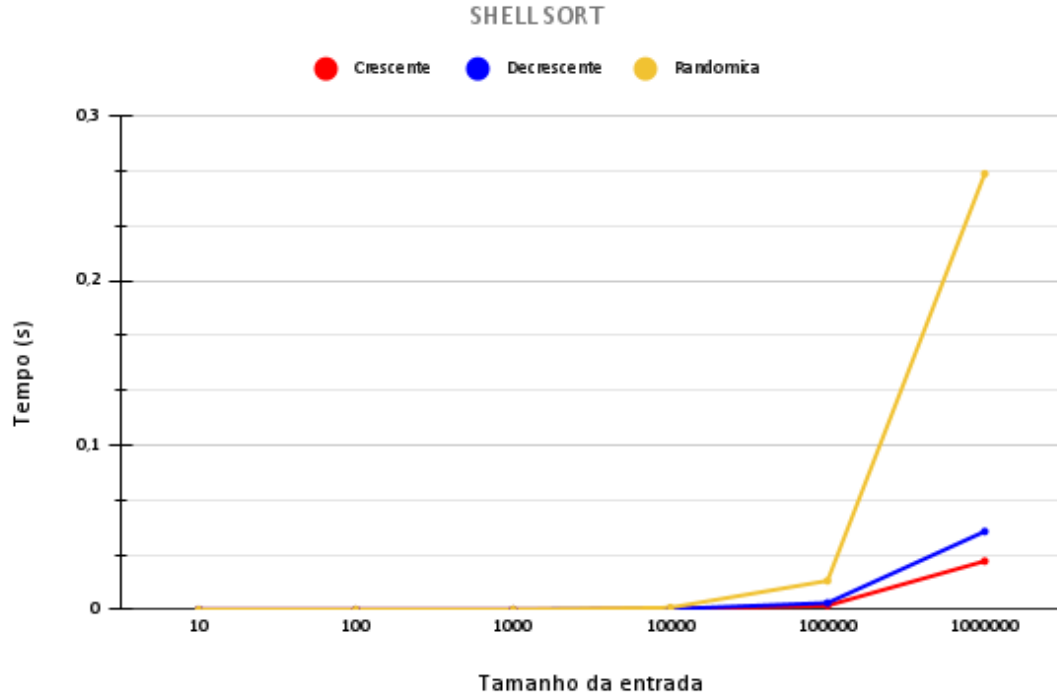


Figura 28: Gráfico de tempo de execução do *Shell-sort*.

De acordo com os dados do gráfico (Figura 28) e da tabela 4, o comportamento do algoritmo se mantém estável para entradas com até 10000 elementos, apresentando diferenças pouco expressivas no tempo de execução. Entretanto, ao ultrapassar esse volume, apesar de não ter uma diferença tão grande, o tempo cresce substancialmente, o que demonstra a influência da complexidade no desempenho. Nessa análise, destacam-se: **melhor caso (ordem crescente)**, **médio caso (ordem decrescente)** e **pior caso (ordem randômica)**.

5.4 *Bubble Sort*

Tabela 5: Tempos de execução em segundos para diferentes entradas (*Bubble-Sort*)

	TAMANHO DA ENTRADA					
TIPO	10	100	1000	10000	100000	1000000
Crescente	0.000003	0.000002	0.000003	0.000016	0.000132	0.001414
Decrescente	0.000002	0.000022	0.002713	0.175286	17.654227	1777.884770
Randômico	0.000004	0.000042	0.001482	0.154889	21.603197	2226.894969

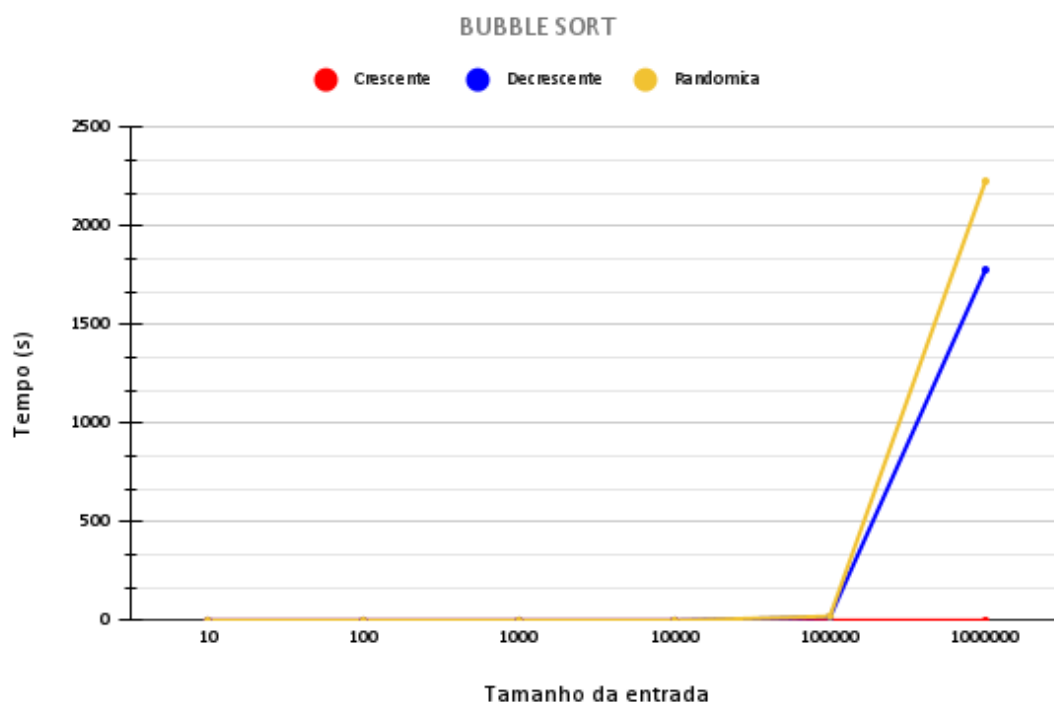


Figura 29: Gráfico de tempo de execução do *Bubble-sort*.

Conforme mostrado no gráfico (Figura 29) e na tabela 5, até o limite de 1000 elementos, as variações no tempo de execução do algoritmo são praticamente irrelevantes. Porém, ao lidar com quantidades maiores, apesar que com 10000 elementos seja uma diferença mínima, o tempo passa a aumentar de forma notável, evidenciando o peso da complexidade na per-

formance. Assim, consideram-se três situações: **melhor caso** (entrada crescente), **caso médio** (entrada decrescente) e **pior caso** (entrada randômica).

5.5 Merge Sort

Tabela 6: Tempos de execução em segundos para diferentes entradas (*Merge-Sort*)

	TAMANHO DA ENTRADA					
TIPO	10	100	1000	10000	100000	1000000
Crescente	0.000002	0.000006	0.000054	0.000714	0.008439	0.097912
Decrescente	0.000004	0.000014	0.000056	0.000735	0.008526	0.096904
Randômico	0.000005	0.000010	0.000095	0.001177	0.014462	0.172827

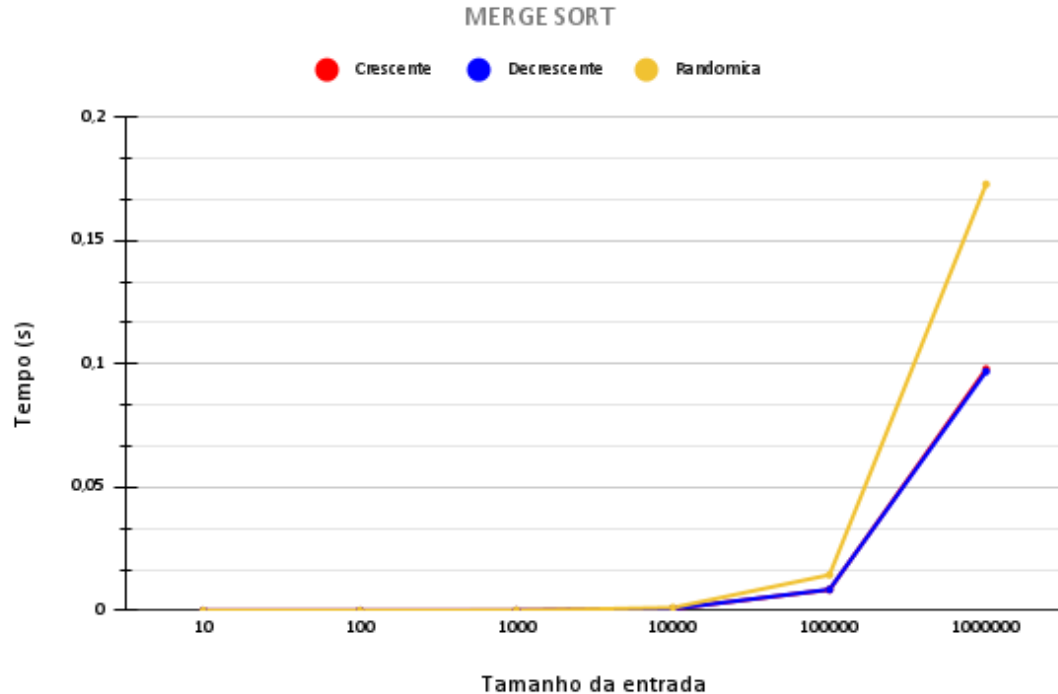


Figura 30: Gráfico de tempo de execução do *Merge-sort*.

De acordo com os dados apresentados no gráfico (Figura 30) e na tabela 6, o algoritmo mantém um desempenho relativamente estável para entradas de até 10.000 elementos, com variações de tempo pouco significativas. No entanto, ao ultrapassar esse número, observa-se um aumento considerável no tempo de execução, evidenciando o impacto da complexidade do algoritmo sobre o desempenho. Para esta análise, são destacados: **melhor caso (vetor em ordem crescente)**, **caso médio (vetor em ordem decrescente)** e **pior caso (vetor em ordem aleatória)**, apesar de não terem uma diferença tão significativa.

5.6 *Quick Sort*

5.6.1 Versão 1 (Primeiro elemento como pivô)

Tabela 7: Tempos de execução em segundos para diferentes entradas (*Quick-Sort*)

	TAMANHO DA ENTRADA					
TIPO	10	100	1000	10000	100000	1000000
Crescente	0.000003	0.000019	0.001948	0.092711	9.241017	926.940333
Decrescente	0.000006	0.000025	0.002345	0.125672	12.643813	1269.682600
Randômico	0.000002	0.000012	0.000106	0.001425	0.011204	0.130757

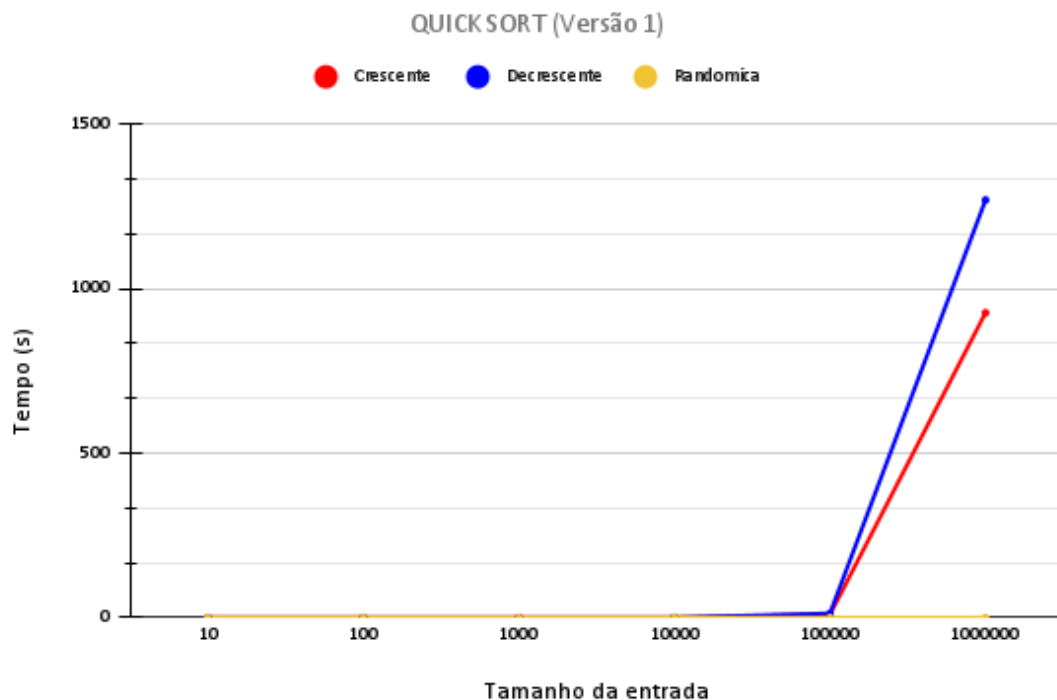


Figura 31: Gráfico de tempo de execução do *Quick-sort* (versão 1).

Como ilustrado no gráfico da Figura 31 e na Tabela 7, o tempo de execução do algoritmo mantém-se praticamente constante para entradas de até 10.000 elementos, apresentando apenas variações mínimas. Entretanto, à medida que o tamanho das entradas ultrapassa esse limite, observa-se um aumento expressivo no tempo de processamento para as entradas crescente e decrescente, evidenciando o impacto da complexidade do algoritmo na sua performance nessas configurações específicas.

Observação importante: nas execuções envolvendo vetores de 1.000.000 de elementos em ordem crescente e decrescente, ocorreu *StackOverflow* devido à profundidade excessiva das chamadas recursivas — comportamento esperado quando o particionamento se torna extremamente desequilibrado. Para contornar essa limitação e permitir a conclusão das medições, utilizou-se o recurso do Linux *ulimit -s unlimited*, que remove temporariamente o limite padrão de tamanho da pilha, possibilitando ao algoritmo finalizar a execução mesmo em cenários desfavoráveis.

5.6.2 Versão 2 (Utilização da média para escolha do pivô)

Tabela 8: Tempos de execução em segundos para diferentes entradas (*Quick-Sort*)

	TAMANHO DA ENTRADA					
TIPO	10	100	1000	10000	100000	1000000
Crescente	0.000003	0.000007	0.000119	0.000557	0.004504	0.051418
Decrescente	0.000005	0.000020	0.000066	0.000788	0.004900	0.056870
Randômico	0.000003	0.000010	0.000216	0.001401	0.011000	0.133150

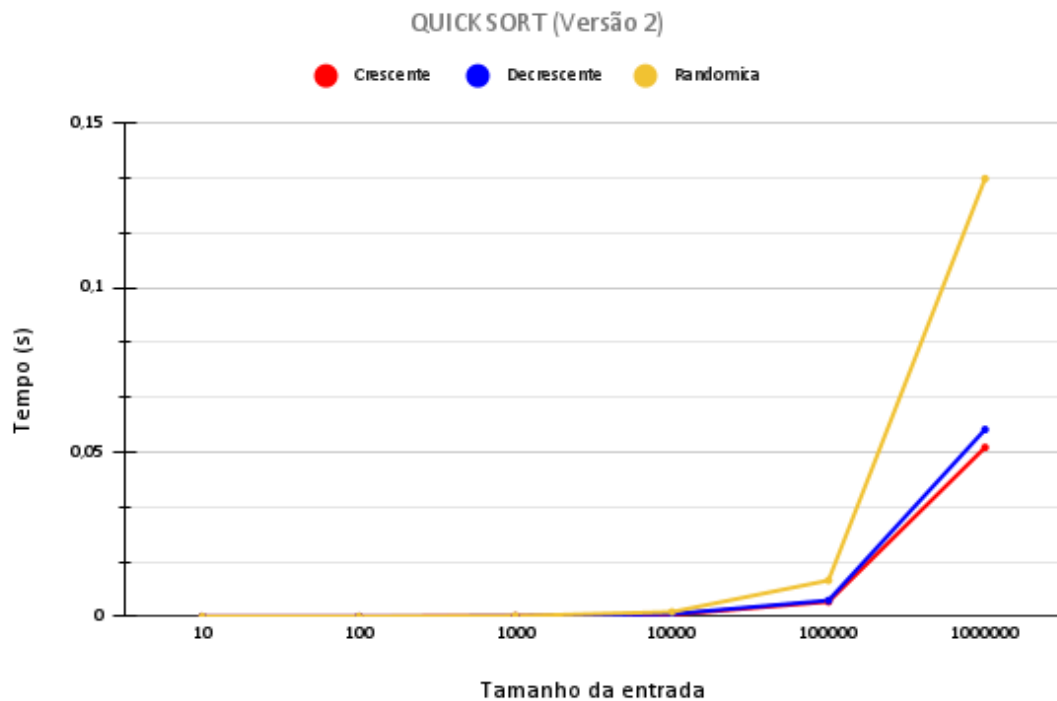


Figura 32: Gráfico de tempo de execução do *Quick-sort* (versão 2).

A análise do gráfico (Figura 32), em conjunto com a Tabela 8, evidencia que, para tamanhos de entrada inferiores a 100.000 elementos, o tempo de execução apresenta pouca variação. À medida que o tamanho do vetor ultrapassa esse limite, observa-se um crescimento gradual no tempo de processamento; porém, essa elevação permanece moderada. Esse

comportamento reforça a eficiência da estratégia de escolha do pivô — baseada na média — que contribui para manter a distribuição das partições equilibrada e, conseqüentemente, garante um desempenho consistente mesmo em entradas maiores.

5.6.3 Versão 3 (Pivô selecionado de forma randômica)

Tabela 9: Tempos de execução em segundos para diferentes entradas (*Quick-Sort*)

	TAMANHO DA ENTRADA					
TIPO	10	100	1000	10000	100000	1000000
Crescente	0.000014	0.000065	0.000583	0.005283	0.037995	0.410742
Decrescente	0.000009	0.000051	0.001239	0.005381	0.035388	0.372553
Randômico	0.000016	0.000066	0.001137	0.004362	0.038673	0.413493

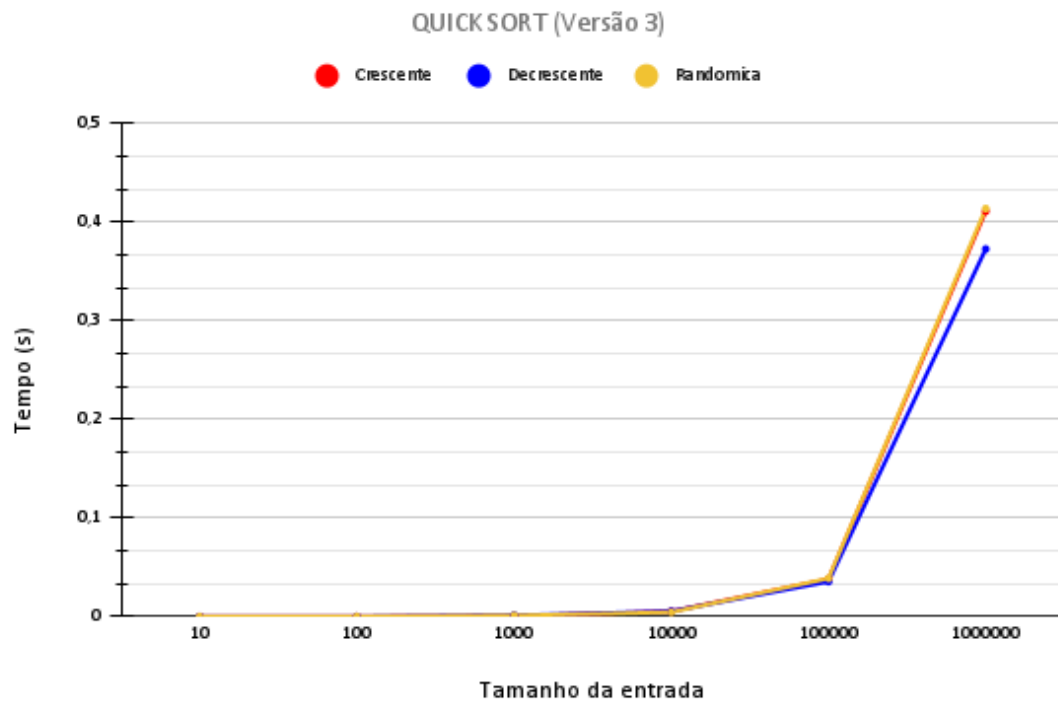


Figura 33: Gráfico de tempo de execução do *Quick-sort* (versão 3).

A interpretação do gráfico da Figura 33, juntamente com os dados apresentados na Tabela 9, mostra que, para entradas com até aproximadamente 10.000 elementos, o tempo de execução do algoritmo permanece bastante estável, com variações mínimas. Quando o tamanho do vetor supera esse valor, nota-se um aumento progressivo no tempo de processamento; ainda assim, esse crescimento ocorre de maneira controlada. Esse resultado demonstra a eficiência da estratégia de seleção aleatória do pivô, que reduz significativamente a chance de partições desbalanceadas e proporciona um desempenho uniforme mesmo em vetores de grande dimensão.

5.7 *Heap Sort (Min-Heap)*

Tabela 10: Tempos de execução em segundos para diferentes entradas (*Heap-Sort*)

	TAMANHO DA ENTRADA					
TIPO	10	100	1000	10000	100000	1000000
Crescente	0.000003	0.000019	0.000395	0.001590	0.017410	0.206366
Decrescente	0.000005	0.000023	0.000266	0.002526	0.018113	0.211586
Randômico	0.000005	0.000027	0.000306	0.001773	0.022837	0.300125

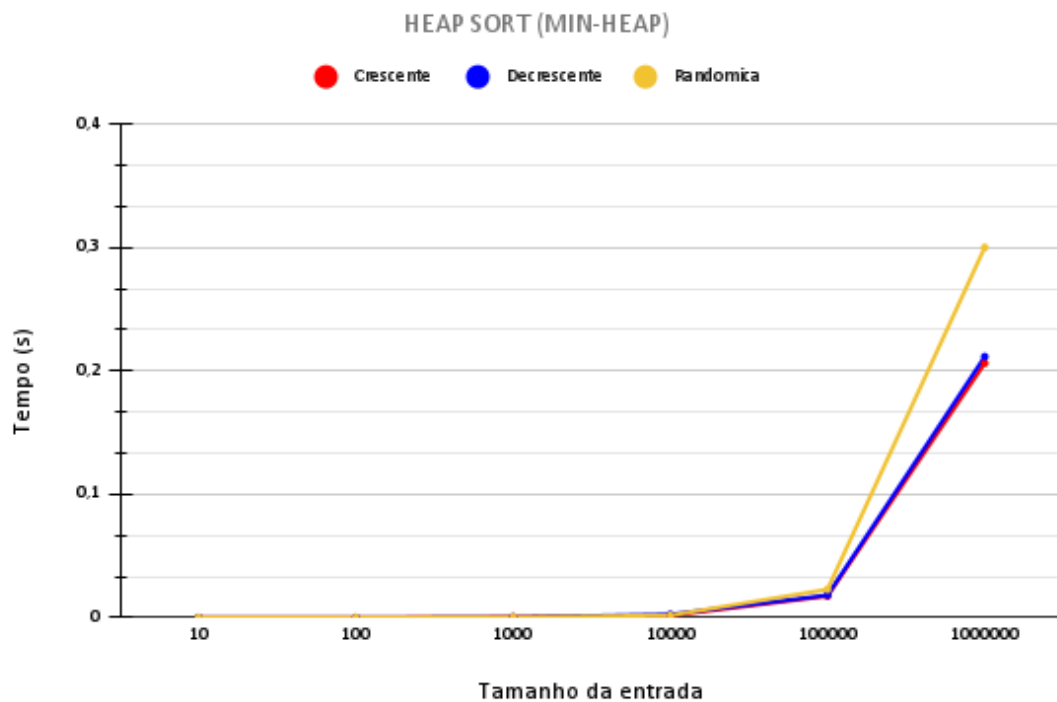


Figura 34: Gráfico de tempo de execução do *Heap-sort*.

De acordo com os dados apresentados no gráfico (Figura 34) e na Tabela 10, o Heap Sort mantém um desempenho bastante estável para tamanhos de entrada inferiores a 10.000 elementos, apresentando variações de tempo pouco expressivas entre os diferentes tipos de ordenação inicial. Esse comportamento era esperado, uma vez que a estrutura de heap reduz significativamente a influência da ordem dos dados sobre o número de operações realizadas.

5.8 Comparação dos Algoritmos

5.8.1 Crescente

Tabela 11: Tempos de execução em segundos para entrada CRESCENTE

	TAMANHO DA ENTRADA					
ALGORITMO	10	100	1000	10000	100000	1000000
Insertion Sort	0.000004	0.000005	0.000011	0.000041	0.000266	0.002666
Selection Sort	0.000002	0.000015	0.001050	0.098688	9.838451	996.422985
Shell Sort	0.000003	0.000003	0.000023	0.000190	0.002412	0.029447
Bubble Sort	0.000003	0.000002	0.000003	0.000016	0.000132	0.001414
Merge Sort	0.000002	0.000006	0.000054	0.000714	0.008439	0.097912
Quick Sort (versão 2)	0.000003	0.000007	0.000119	0.000557	0.004504	0.051418
Heap Sort (Min-Heap)	0.000003	0.000019	0.000395	0.001590	0.017410	0.206366

Crescente

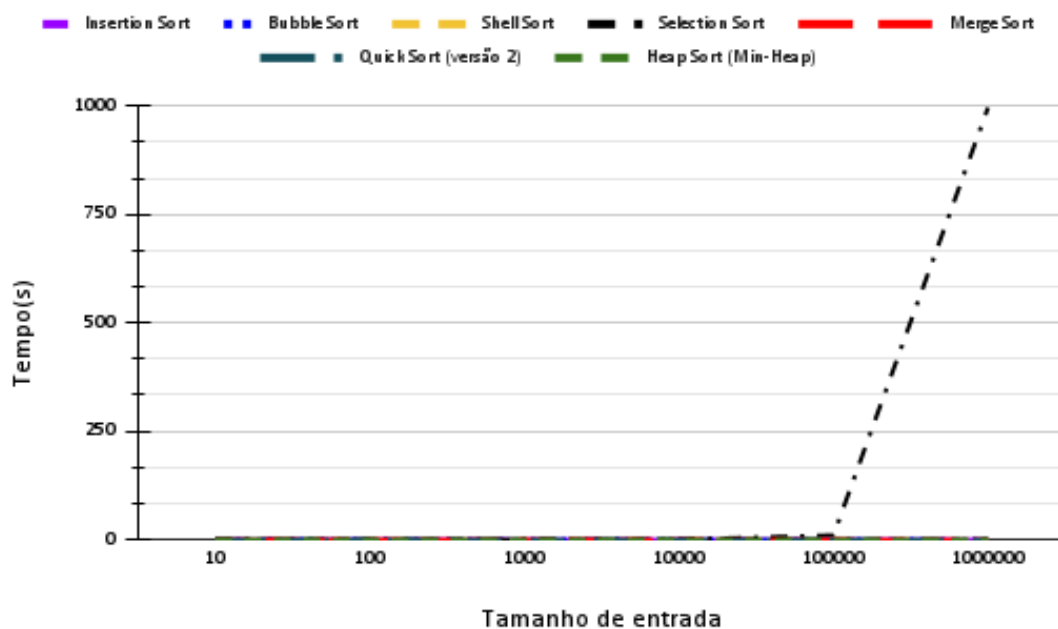


Figura 35: Gráfico de tempo de execução para entrada CRESCENTE.

Conforme evidenciado pelo gráfico (Figura 35) e pela tabela 11 apresentados, o algoritmo *Selection Sort* apresenta um aumento significativo no tempo de execução a partir de entradas superiores a 10.000 elementos, em razão de sua complexidade e da lógica inerente ao seu funcionamento.

5.8.2 Decrescente

Tabela 12: Tempos de execução em segundos para entrada DECRESCENTE

	TAMANHO DA ENTRADA					
ALGORITMO	10	100	1000	10000	100000	1000000
Insertion Sort	0.000002	0.000017	0.001514	0.106543	9.308213	1109.132003
Selection Sort	0.000002	0.000026	0.002159	0.191703	19.514748	1992.846278
Shell Sort	0.000003	0.000006	0.000027	0.000348	0.004011	0.047476
Bubble Sort	0.000002	0.000022	0.002713	0.175286	17.654227	1777.884770
Merge Sort	0.000004	0.000014	0.000056	0.000735	0.008526	0.096904
Quick Sort (versão 2)	0.000005	0.000020	0.000066	0.000788	0.004900	0.056870
Heap Sort (Min-Heap)	0.000005	0.000023	0.000266	0.002526	0.018113	0.211586

Decrescente

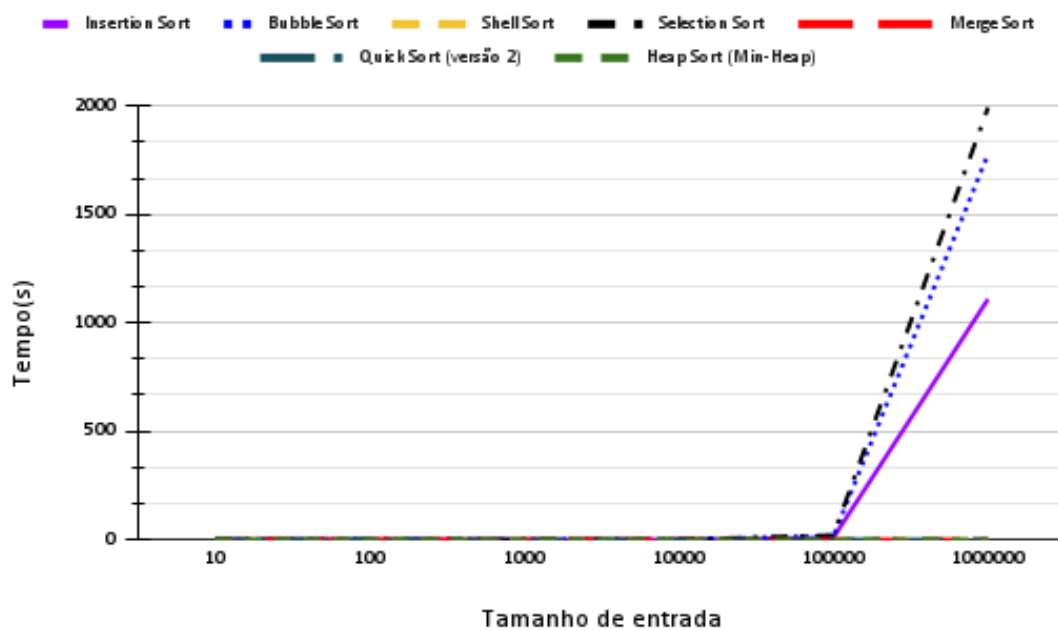


Figura 36: Gráfico de tempo de execução para entrada DECRESCENTE.

A análise mostra que, para entradas de até 1000 elementos, os algoritmos exibem desempenhos próximos. No entanto, quando esse limite é ultrapassado, o *Shell Sort* demonstra maior eficiência, preservando tempos reduzidos, ao passo que os algoritmos *Insertion Sort*, *Selection Sort* e *Bubble Sort* apresentam elevação significativa no tempo de execução.

5.8.3 Randômico

Tabela 13: Tempos de execução em segundos para entrada RANDÔMICA

	TAMANHO DA ENTRADA					
ALGORITMO	10	100	1000	10000	100000	1000000
Insertion Sort	0.000002	0.000026	0.000544	0.046389	4.776612	567.916833
Selection Sort	0.000002	0.000029	0.003441	0.275458	29.332271	2922.656336
Shell Sort	0.000002	0.000014	0.000079	0.001203	0.017492	0.265013
Bubble Sort	0.000004	0.000042	0.001482	0.154889	21.603197	2226.894969
Merge Sort	0.000005	0.000010	0.000095	0.001177	0.014462	0.172827
Quick Sort (versão 2)	0.000003	0.000010	0.000216	0.001401	0.011000	0.133150
Heap Sort (Min-Heap)	0.000005	0.000027	0.000306	0.001773	0.022837	0.300125

Randômico

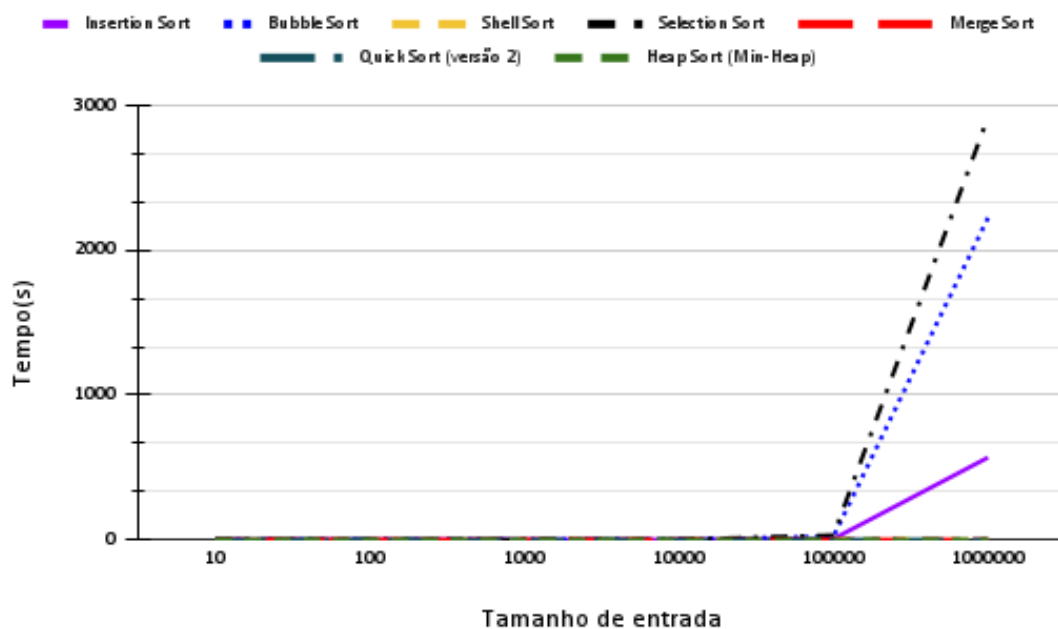


Figura 37: Gráfico de tempo de execução para entrada RANDÔMICO.

Conforme observado, até 1000 elementos os algoritmos apresentam tempos próximos um do outro. A partir de 10.000 elementos, o *Quick Sort (versão 2)* se destaca por manter o menor tempo de execução, enquanto *Insertion Sort*, *Selection Sort* e *Bubble Sort* passam a apresentar crescimento expressivo, especialmente para 100.000 e 1.000.000 elementos, evidenciando o impacto de suas maiores complexidades.

6 Conclusão

A partir das análises realizadas (teóricas, empíricas e gráficas) foi possível compreender de forma sólida o comportamento dos algoritmos de ordenação estudados. Os resultados experimentais confirmam as previsões matemáticas: algoritmos quadráticos, como *Bubble Sort*, *Insertion Sort* e *Selection Sort*, apresentam tempos de execução significativamente maiores à medida que o tamanho da entrada cresce, especialmente para vetores em ordem decrescente.

Em contraste, os algoritmos $O(n \log n)$, como *Merge Sort*, *Quick Sort* e *Heap Sort*, mantêm desempenho muito superior para entradas maiores. Em especial, o *Quick Sort* demonstrou excelente desempenho quando o pivô foi selecionado por estratégias adequadas (média ou randomização), evitando problemas clássicos de degradação para $O(n^2)$. O *Merge Sort* manteve estabilidade em todos os cenários, e o *Heap Sort* apresentou um comportamento consistente entre os casos testados, confirmando sua eficiência na prática.

Também ficou evidente, através dos gráficos e tabelas, como a organização inicial dos dados afeta diretamente os algoritmos quadráticos, enquanto os algoritmos de divisão e conquista ou baseados em *heap* são muito menos sensíveis a essa variabilidade.

Dessa forma, conclui-se que a escolha do algoritmo de ordenação deve levar em conta não apenas a complexidade assintótica, mas também características da entrada, disponibilidade de memória, estabilidade desejada e custo de implementação. O trabalho permitiu não apenas comparar os algoritmos, mas entender profundamente o porquê de seus comportamentos, fortalecendo a base teórica e prática necessária para aplicar técnicas de ordenação em cenários reais de programação.

Referências

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Algoritmos-Teoria e Prática*. MIT Press, 3 edition, 2009.
- [2] Donald E. Knuth. *The Art of Computer Programming (TAOCP)*. 3 edition, 1998.
- [3] Nivio Niviani. *Projeto de Algoritmos com Implementação em Pascal e C*. 3 edition, 2010.
- [4] Robert Sedgewick and Philippe Flajolet. *An Introduction to the Analysis of Algorithms*. 1996.